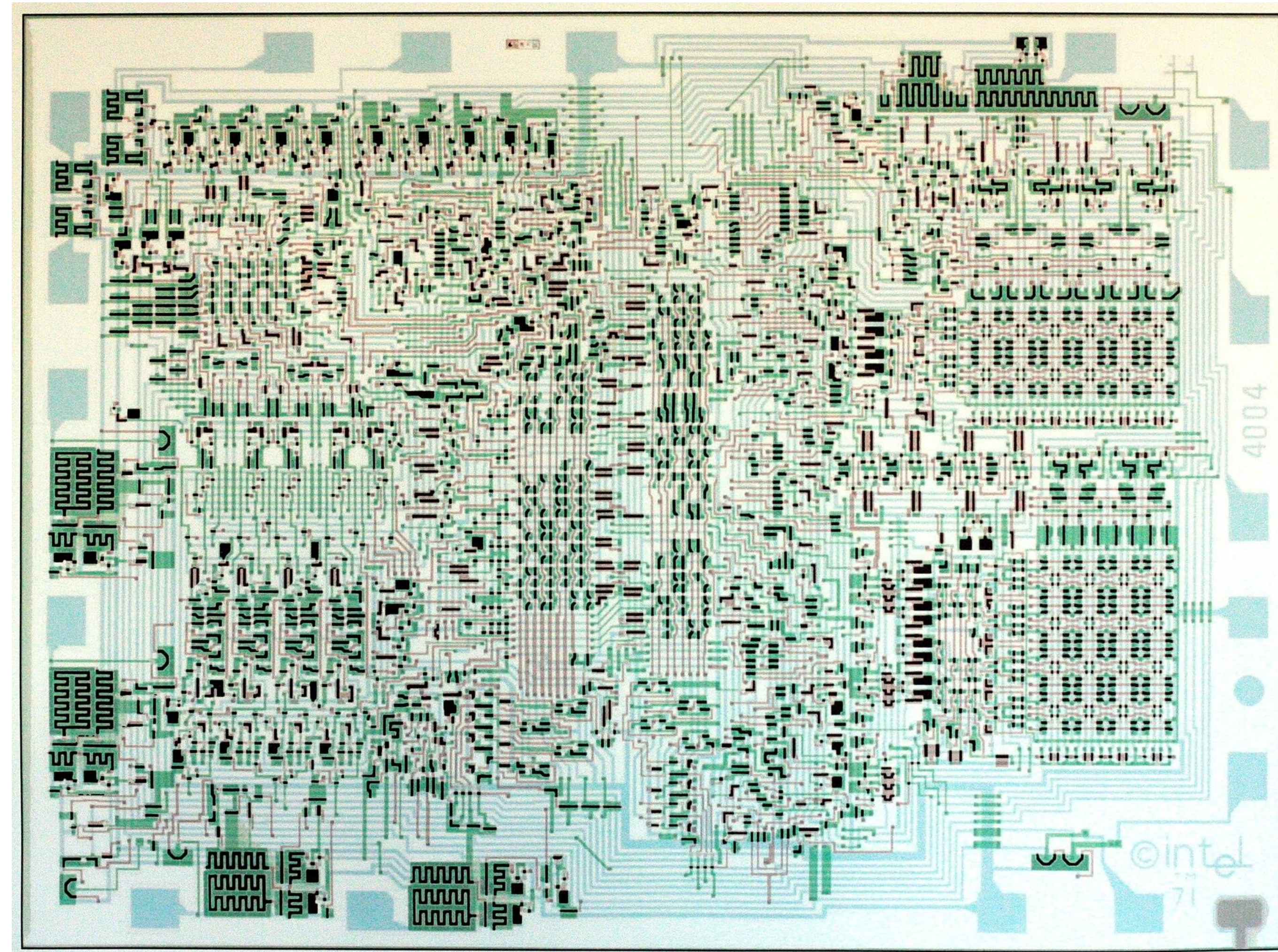


Agentic AI for Chip Design

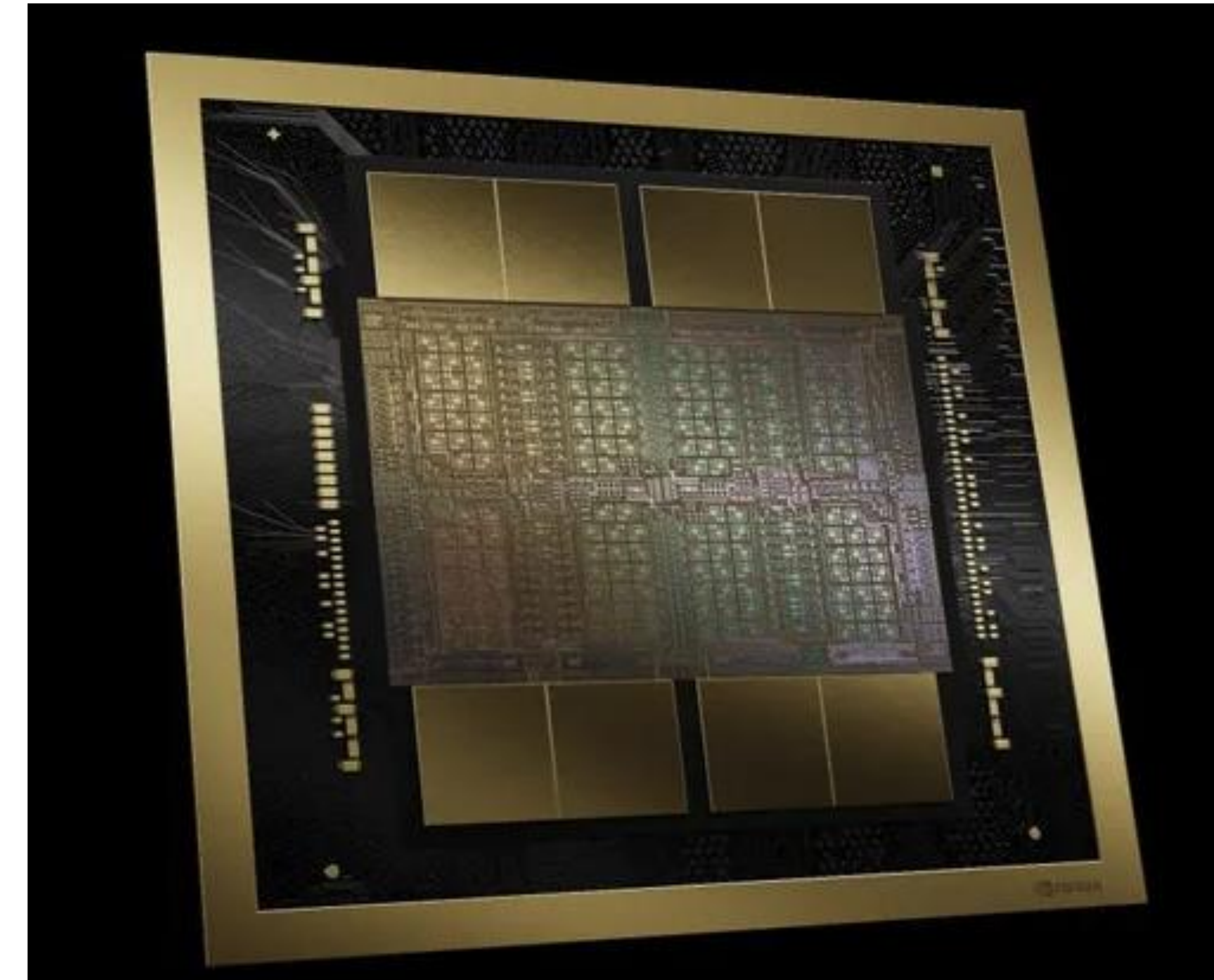
Agentic Design Automation (ADA) toward Autonomous Design

Mark Ren
Founder and CEO, AGENTRYS

Chip Design: Million X Productivity Improvement in Half Century



Intel 4004 (1971)
2300 transistors
100K 4B instruction/sec



NVIDIA B200 (2024)
208 Billion Transistors
20 Peta FP4/sec

1st EDA Driving Force: Increasing Automation and Scaling

1973: SPICE, **automate** circuit simulation

1986: Synopsys Design Compiler, **automate** logic synthesis

1989: Cadence Verilog-XL, **automate** RTL simulation

1997: Cadence Silicon Ensemble , **automate** placement & route

1998: Synopsys PrimeTime , **automate** static timing analysis

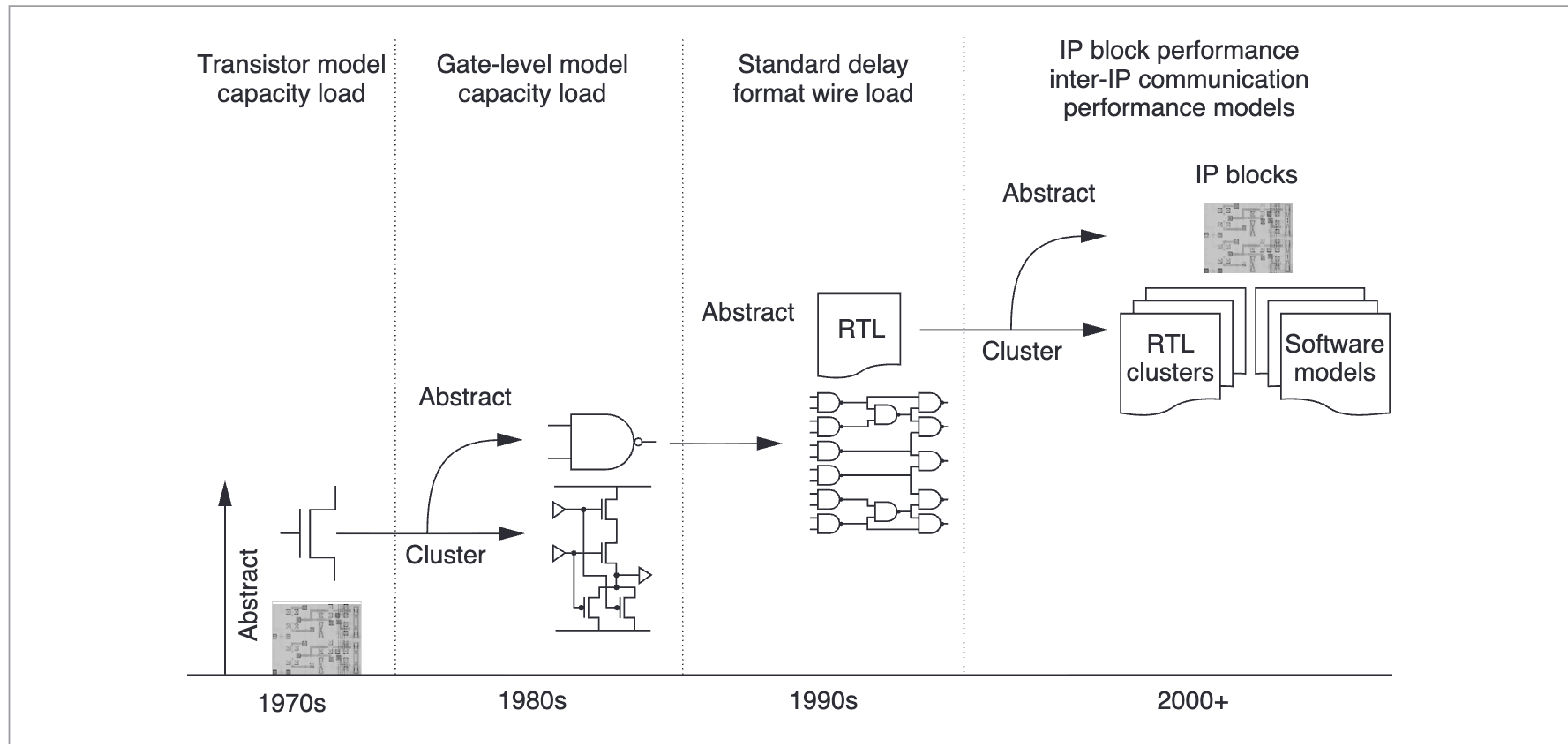
2004: Apache RedHawk, **automate** power grid analysis, scaling with multi-threading

2008: Synopsys PrimeTime , **scaling** STA with multi-threading

2012: Cadence Innovus, **scaling** P&R with extensive multi-threading

2016: Ansys RedHawk-SC, **scaling** power grid analysis with distributed many cores

2nd EDA Driving Force: Raising Level of Abstraction



AI Enables More Automation

Generating insights, knowledge, and ideas. Answer questions about designs, infrastructures, tools, flows, HW domains, etc.

Understanding

Generating design and verification code (RTL, SVA, testbenches, EDA scripts, tools scripts, and configs)

Coding

Summarization, report/log analysis, visualization of design and related data, check rule violations, etc.

Analysis

Triage and fix a design problem, e.g. regression failures for logic design and timing failures for physical design.

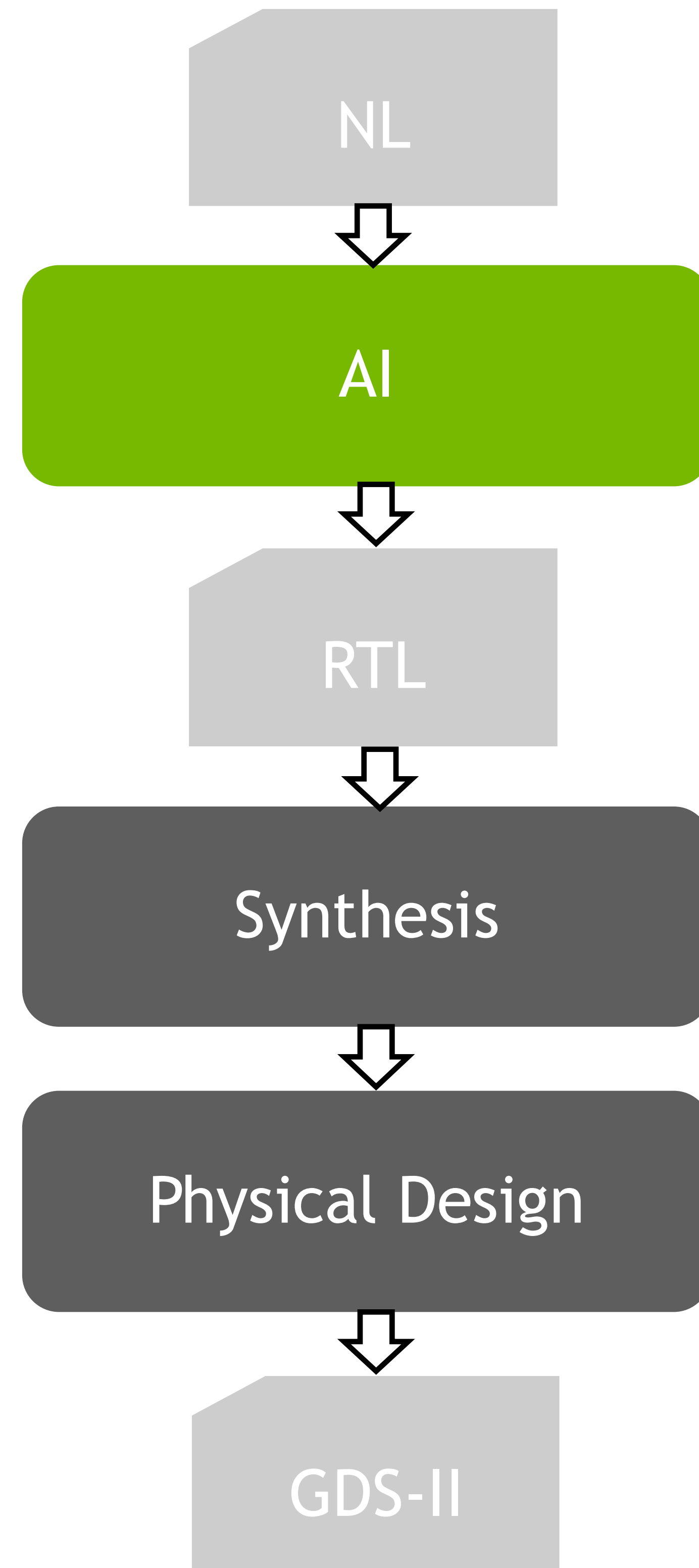
Debug

Optimize for design and verification: coverage closure, recipe optimization, architecture exploration, etc.

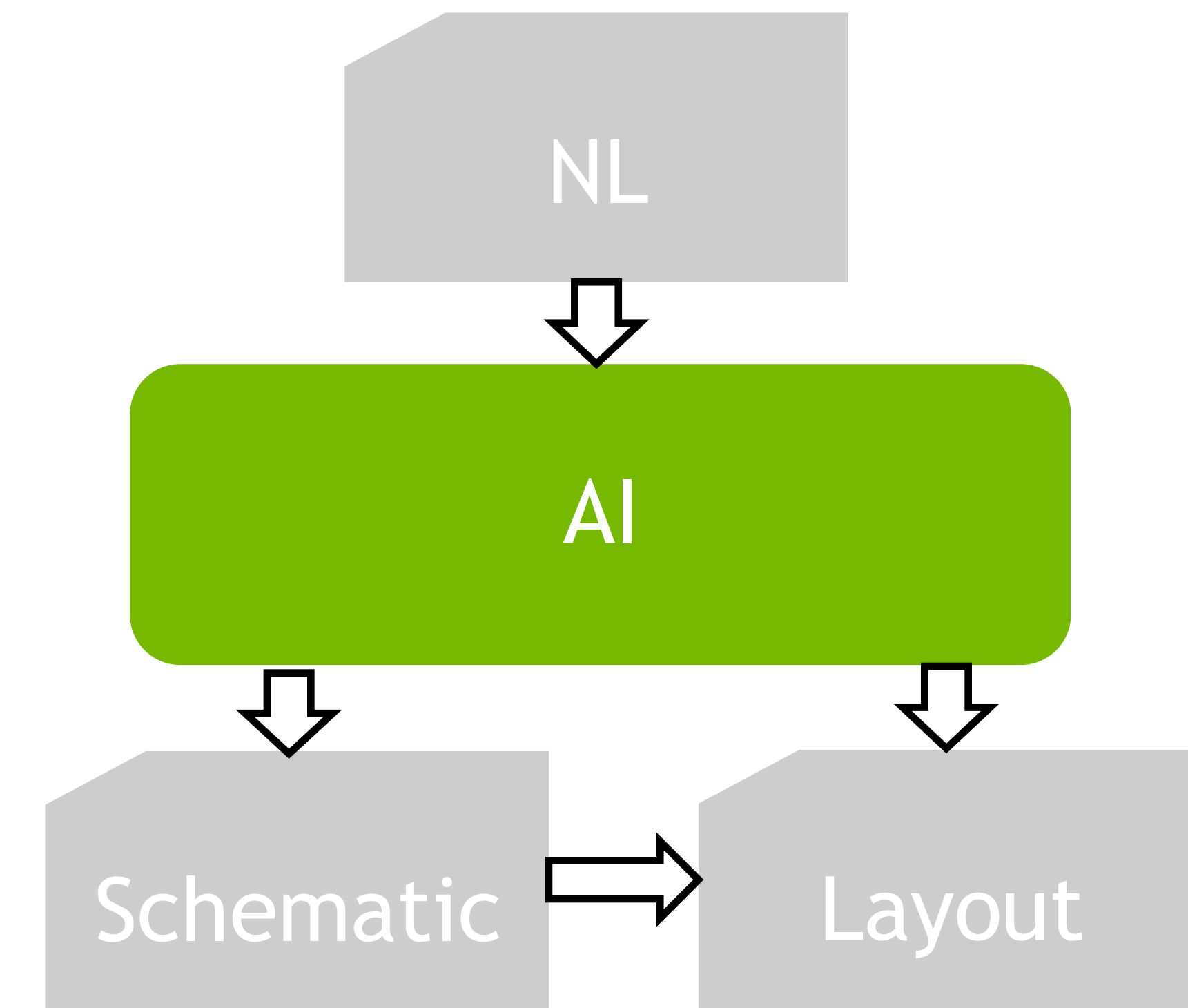
Optimization

AI Raises Design Abstraction Level

Digital

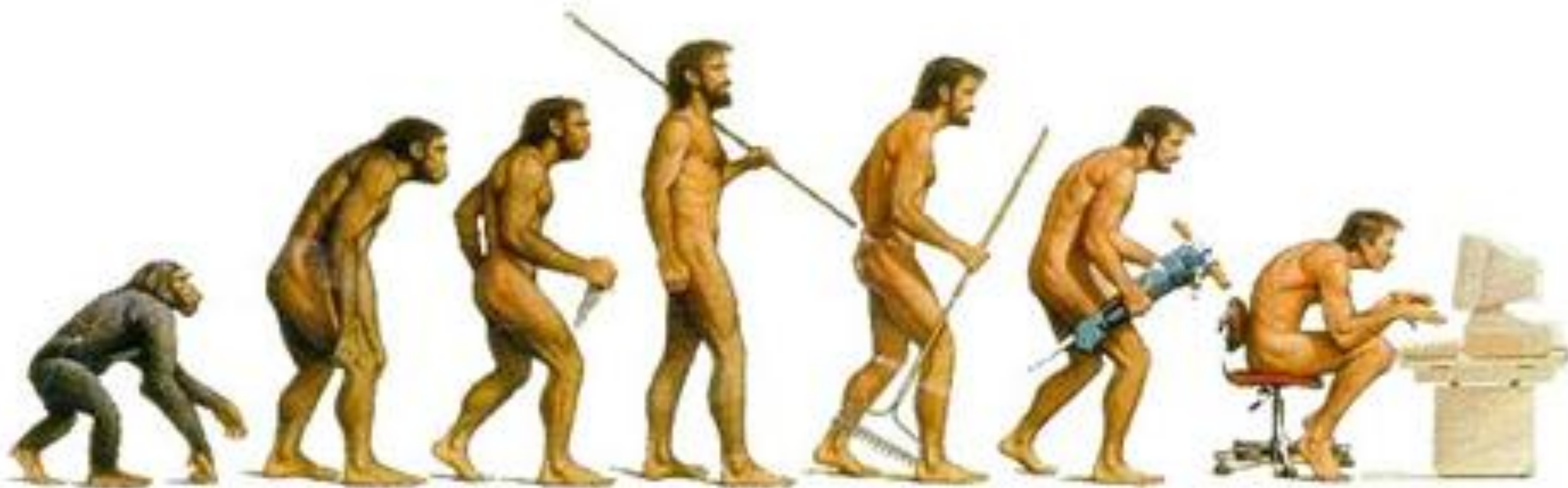


Analog



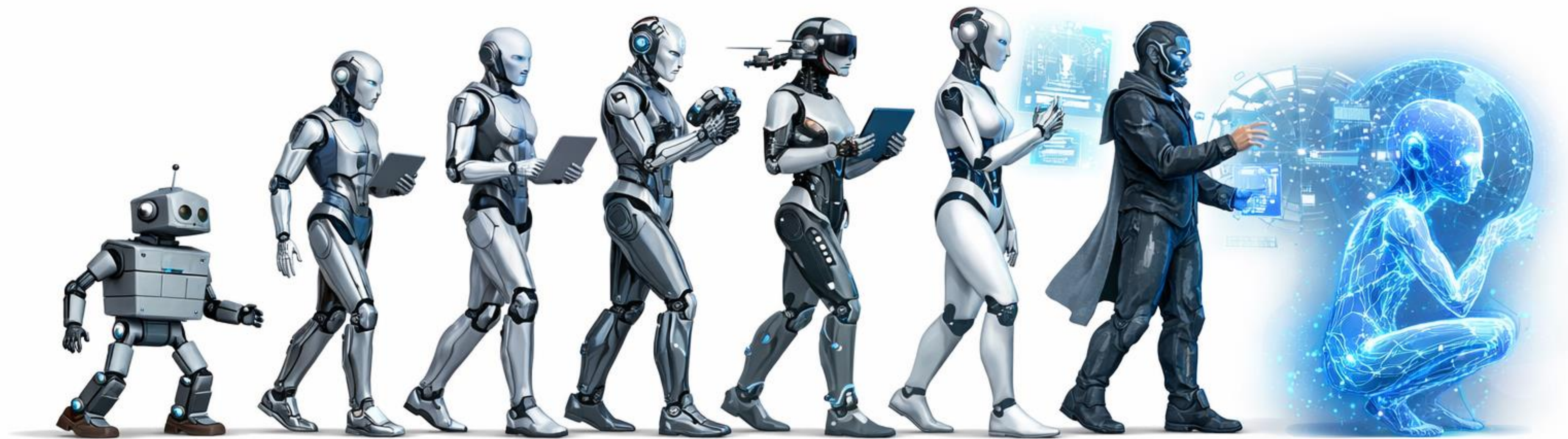
Human Evolution with Tools

Power of human comes from tools we use



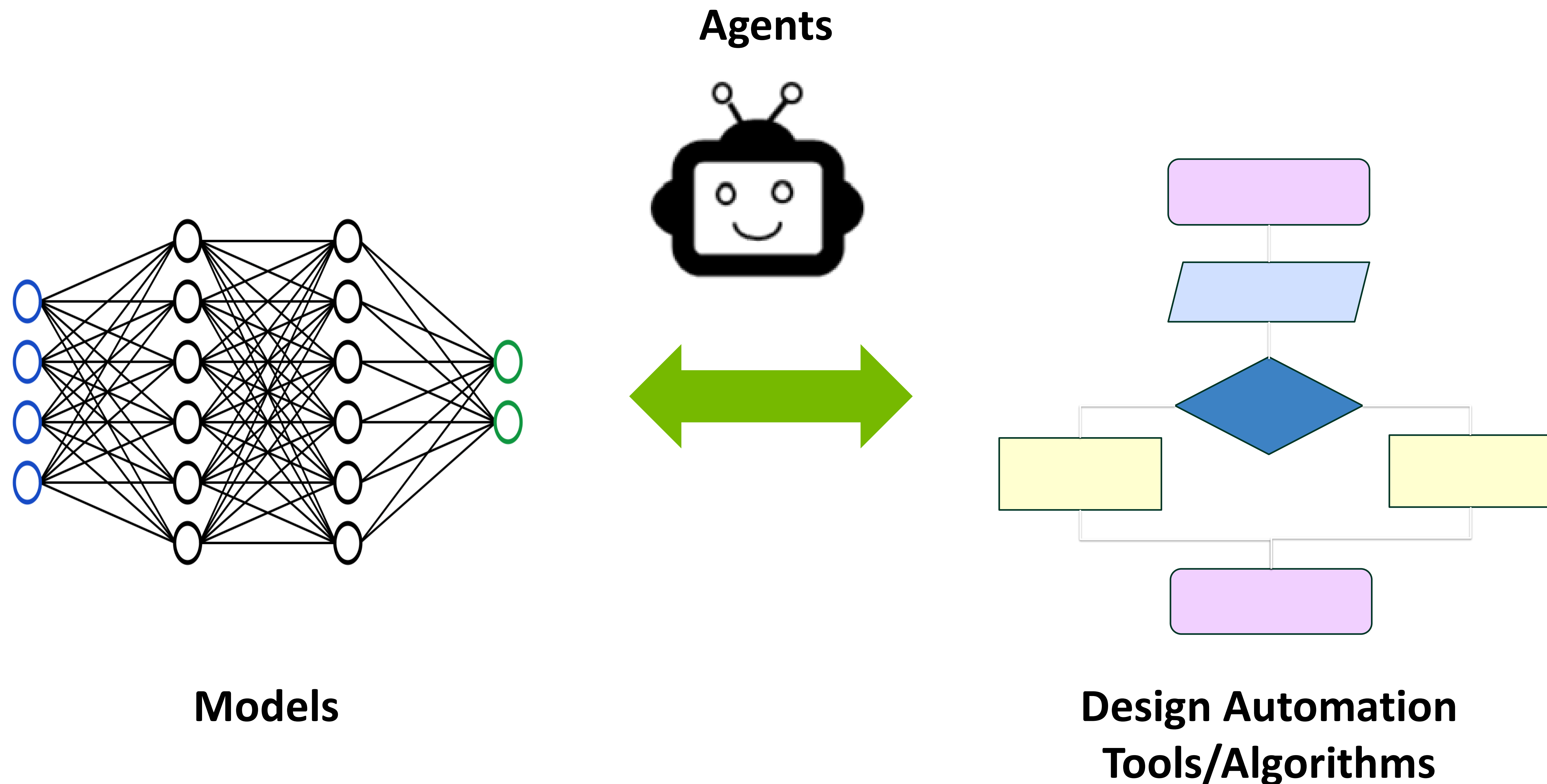
Agent Evolution with Tools

Power of Agents will come from tools they use



Agentic Design Automation (ADA)

Scale the Power of AI and Agent with Design Automation Tools/Algorithms



ADA Example 1: VerilogCoder

Unit Level Spec to RTL Code Generation



Problem Description

Query Module: Implement the following Moore state machine ...

state	(output)	--input--> next state

S	()	--d=0--> S
S	()	--d=1--> S1
S1	()	--d=0--> S
S110	()	--d=0--> S
S110	()	--d=1--> B0
(... some content omitted)		
Wait	(done=1)	--ack=0--> Wait
Wait	(done=1)	--ack=1--> S

Task List

Task: Define the Module Interface



Task: Implement the combinational logic for the S_next signal.



Task: Check and correct the functionality

Verilog

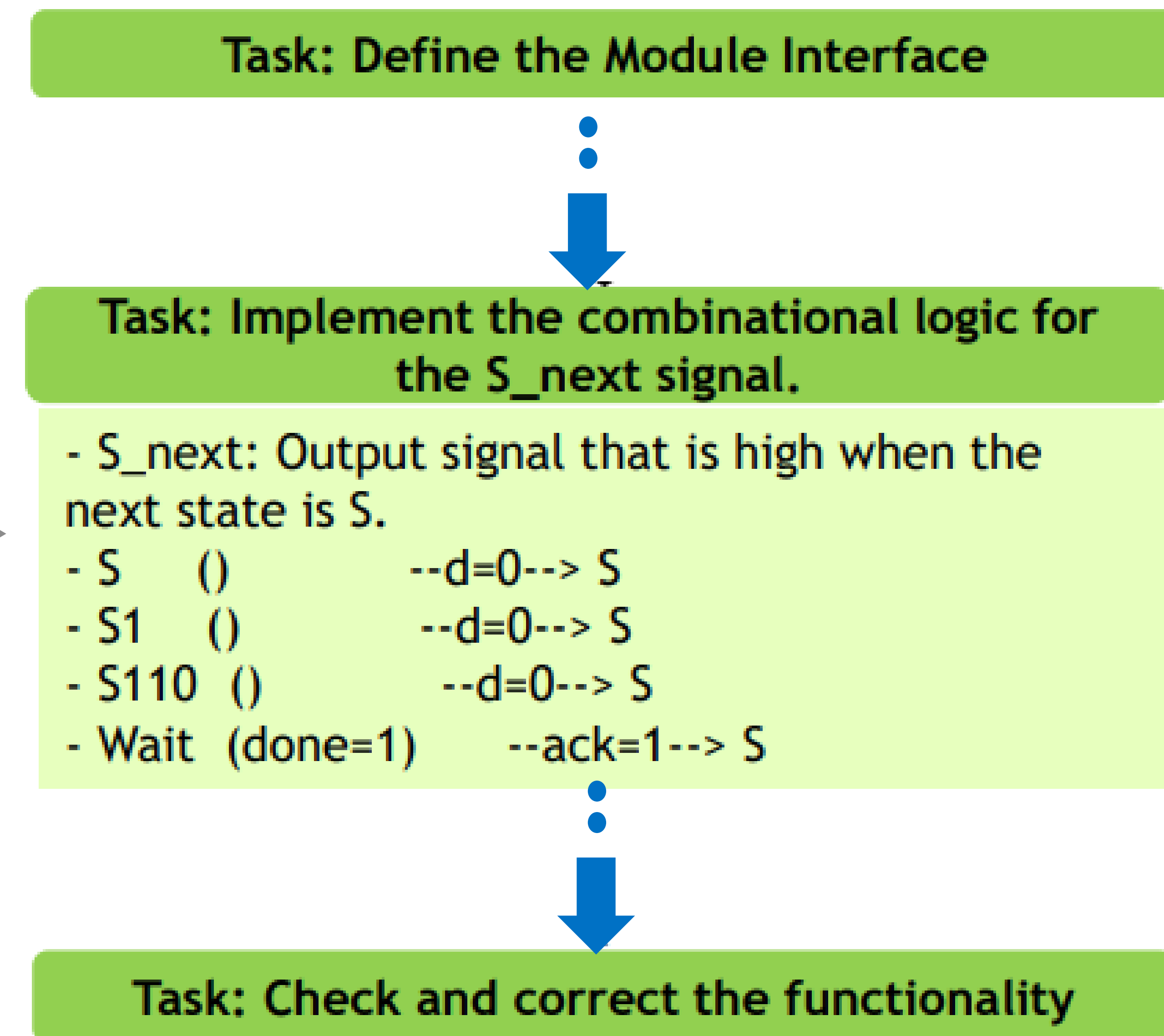
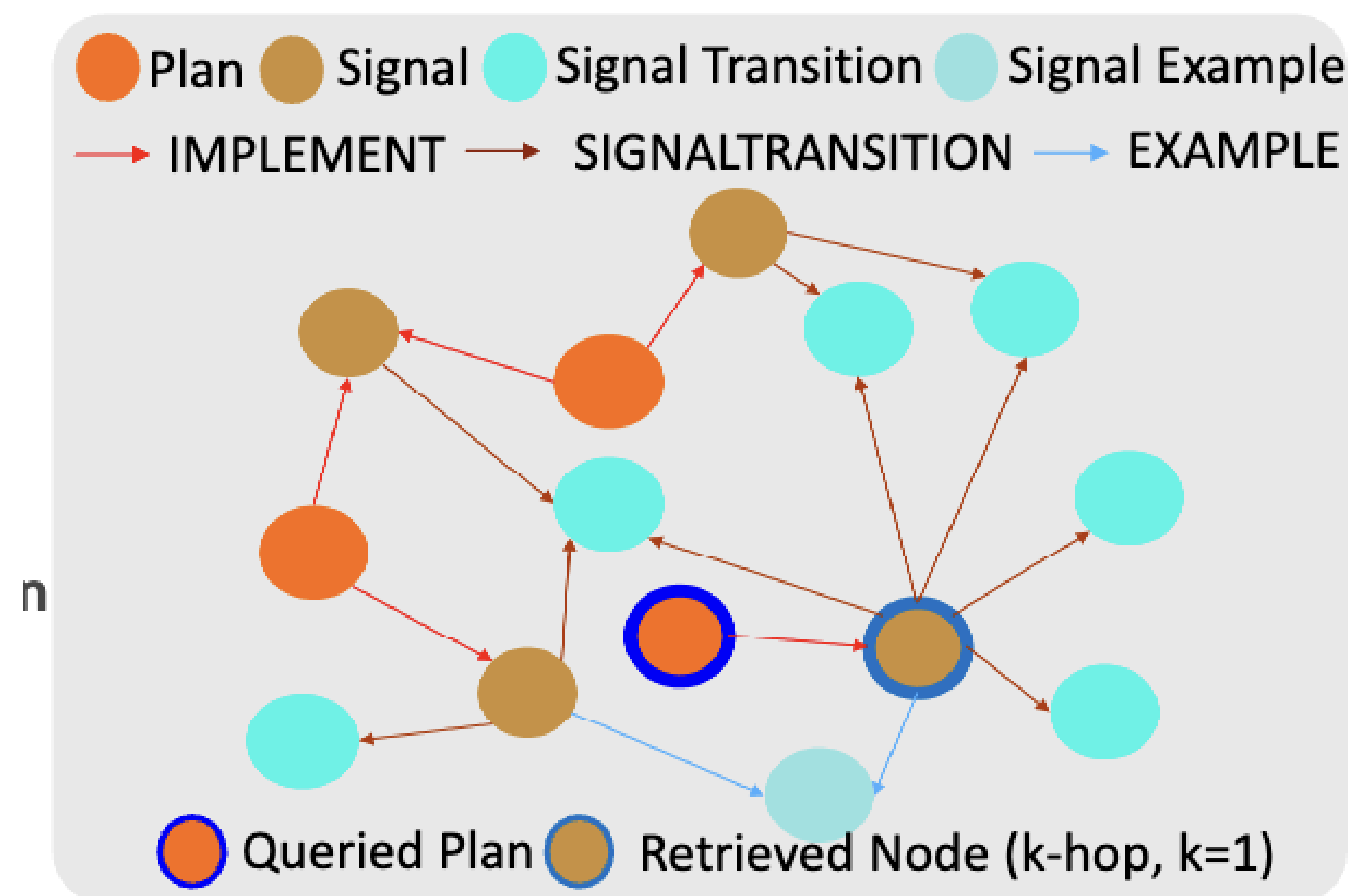
Original Plan missing detailed implementation information

ADA Example 1: VerilogCoder

Tools extract relevant context for sub-task plan

Task Planning Tools

Task-Driven Circuit Relation Graph (TCRG)



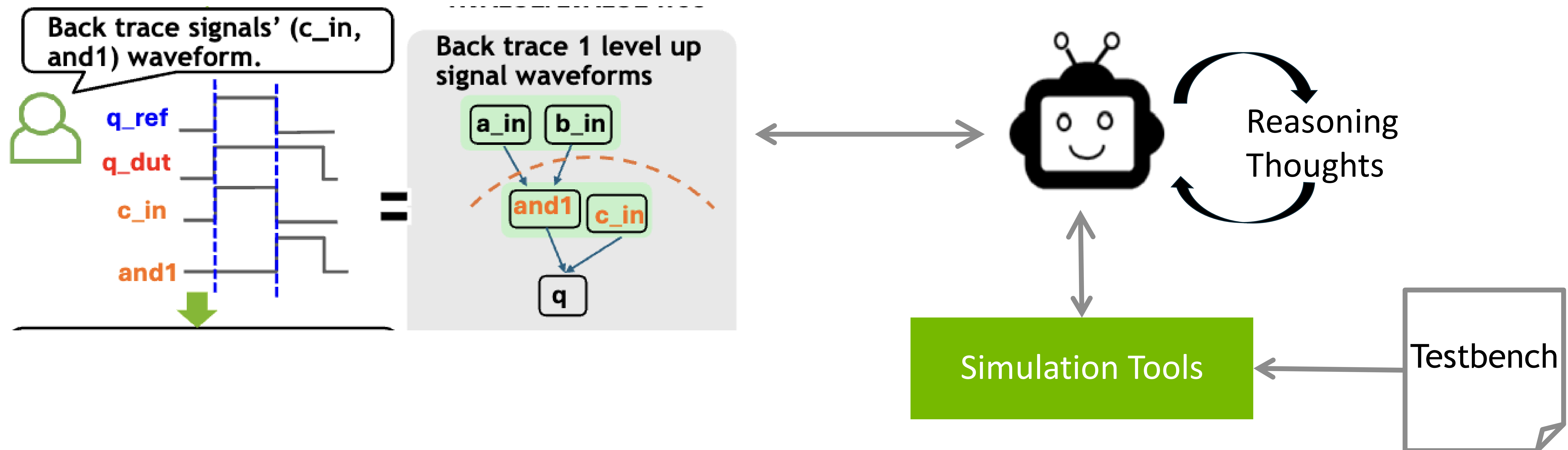
ADA Example 1: VerilogCoder

Tools process relevant context for debug

Code Implementation Tools

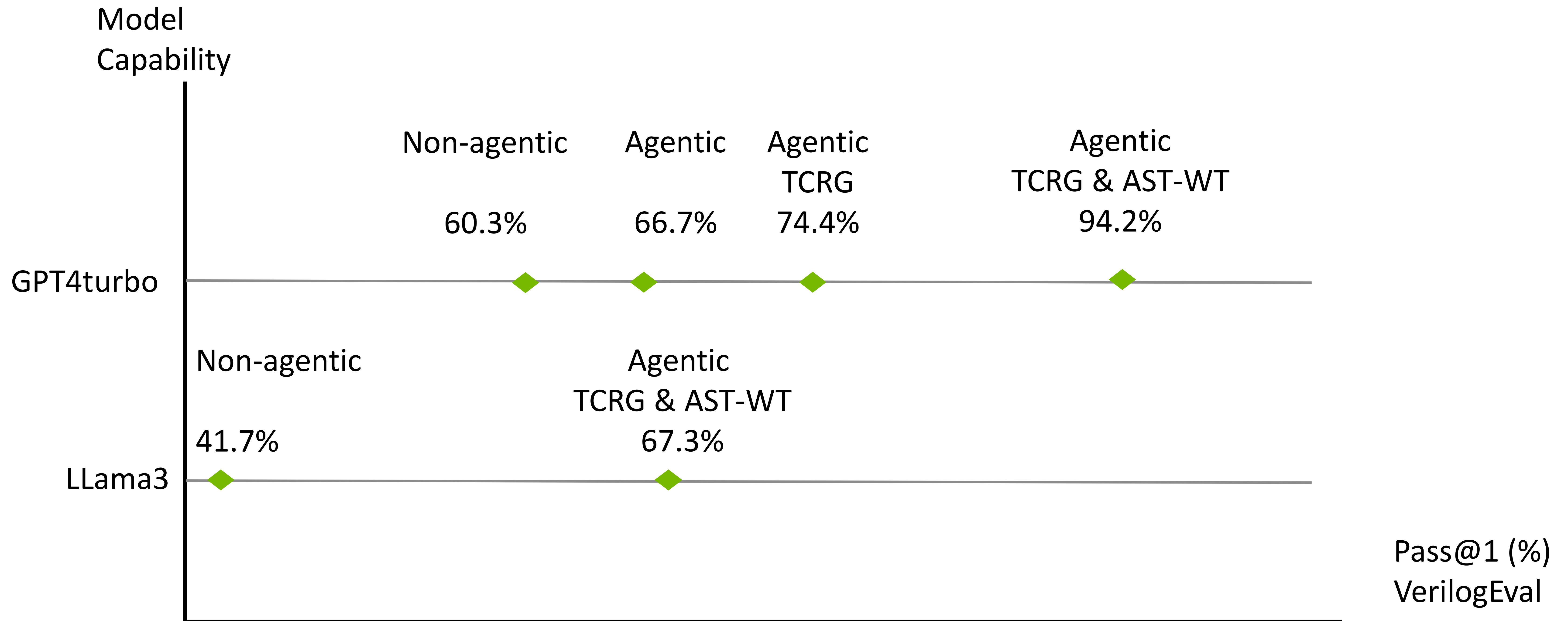
AST-guided waveform debugging tool

Debug



VerilogCoder Agent Solves Most of Problems in VerilogEval

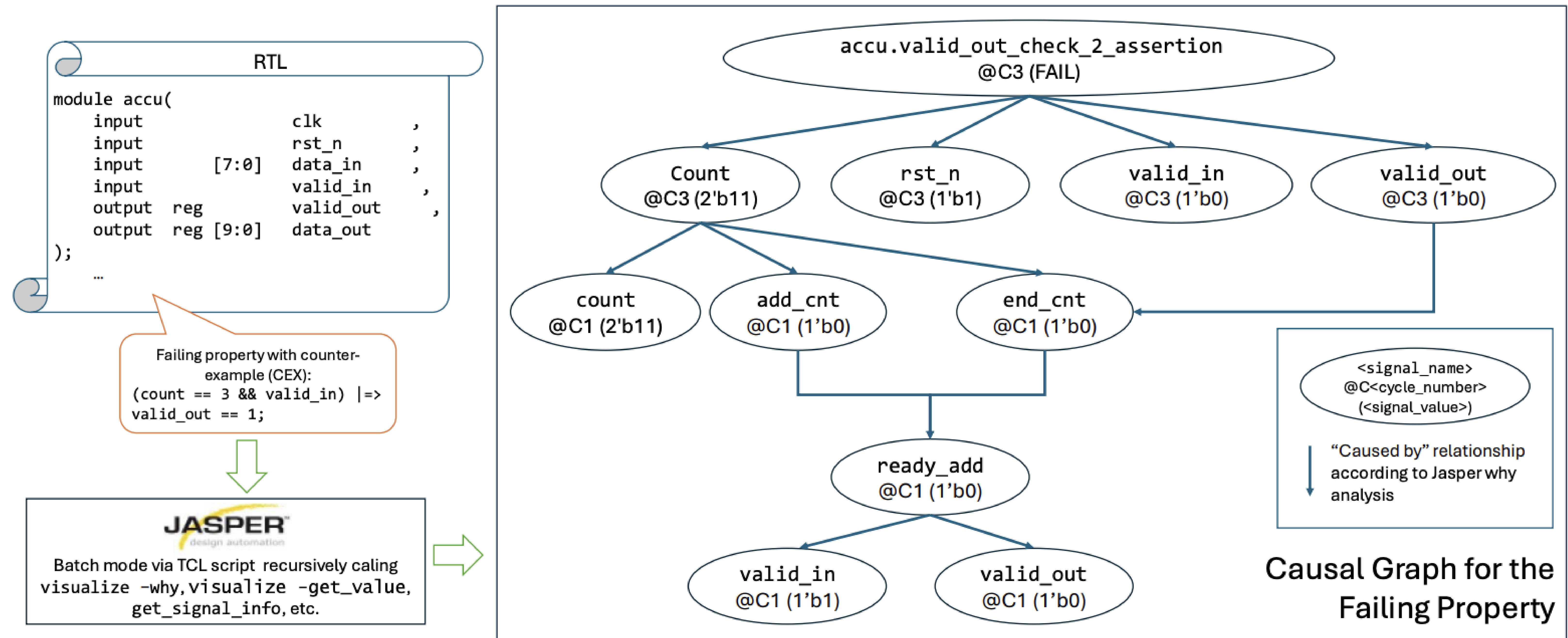
Tools boost agent performance significantly



ADA Example 2: Verification Debug

Tools provide context information for debugging

Debug with **causal graph** generated by EDA tools



ADA Example 2: Verification Debug

Agent reasons with the causal graph

Agent traverse graph to locate RTL and reasoning about the bug; then propose debug hypothesis

Case Study on accumulator

```
## Buggy Line
assign ready_add = valid_out | !valid_in;
## Fixed Line
assign ready_add = !valid_out | valid_in;
```

Golden Answer

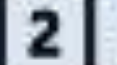
 **Ranked Hypotheses**

Hypothesis 1: Ready_add Logic Issue Preventing valid_out Assertion

Confidence: 48.0%

 **Hypothesis**

The valid_out signal remains low because the ready_add condition (ready_add = valid_out OR !valid_in) does not become true when count reaches 3, causing end_cnt to never trigger a valid_out high; this indicates an RTL bug where the handshake logic for ready_add is flawed.

 **Evidence**

Supporting Evidence:

- The RTL code 'assign end_cnt = ready_add && (count == 'd3)' implies that even if count reaches 3, end_cnt depends on ready_add being true. This supports the hypothesis if ready_add remains low, preventing valid_out from becoming high.

...

 **Causal Chain Timeline**

...

3. Cycle 1 

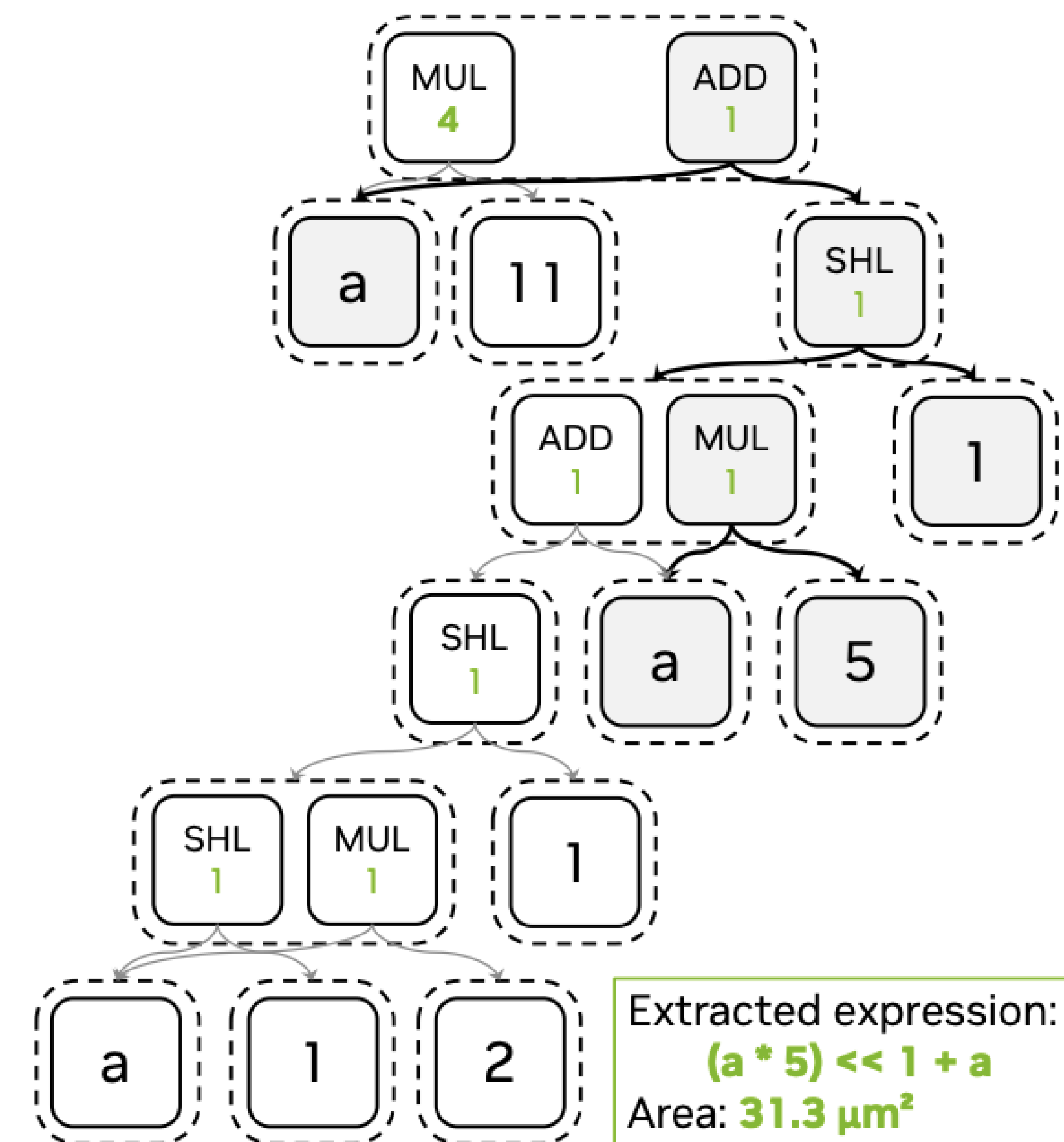
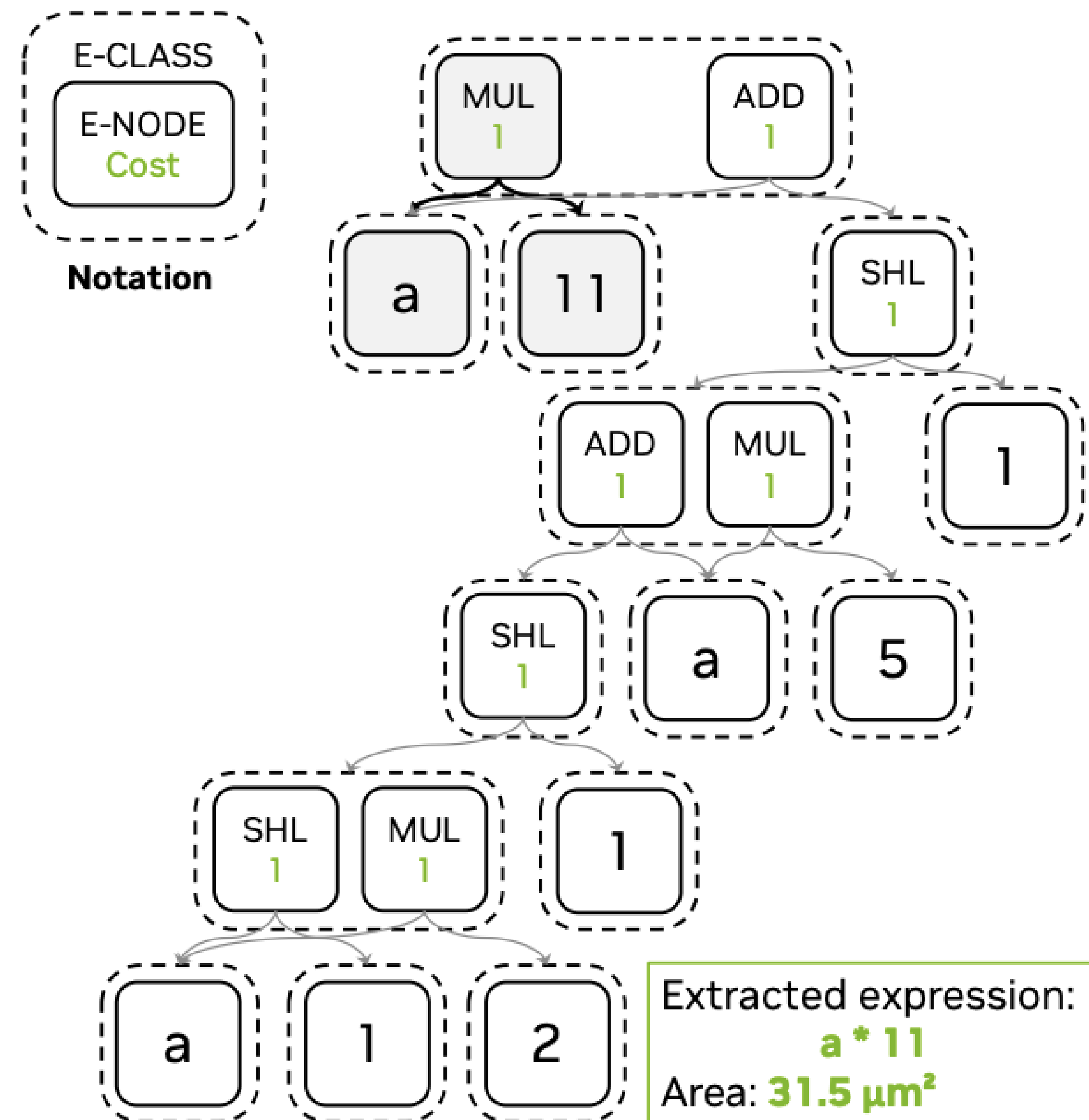
At cycle 1, with valid_in high, the node 'ready_add' evaluates to 1'b0 because it is computed as valid_out OR !valid_in. Since valid_in is high, !valid_in is false and valid_out is low, which blocks accumulation and consequently prevents the triggering of end_cnt and the high assertion of valid_out.

ADA Example 3: RTL Optimization

Leverage e-graph tools

Leverage **e-graph** for correct-by-construction RTL rewriting with E-Graph

E-graph rewriting tools executes the RTL optimization.



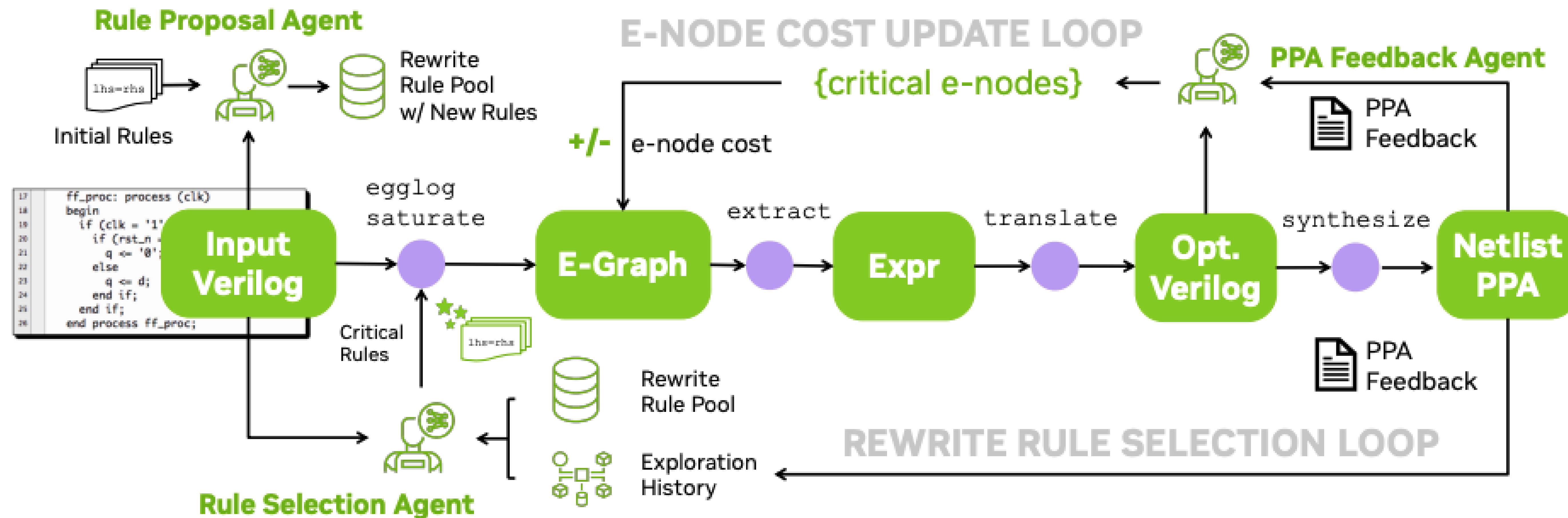
ADA Example 3: RTL Optimization

Agent drives the e-graph rewrite tools

LLM assisted PPA feedback

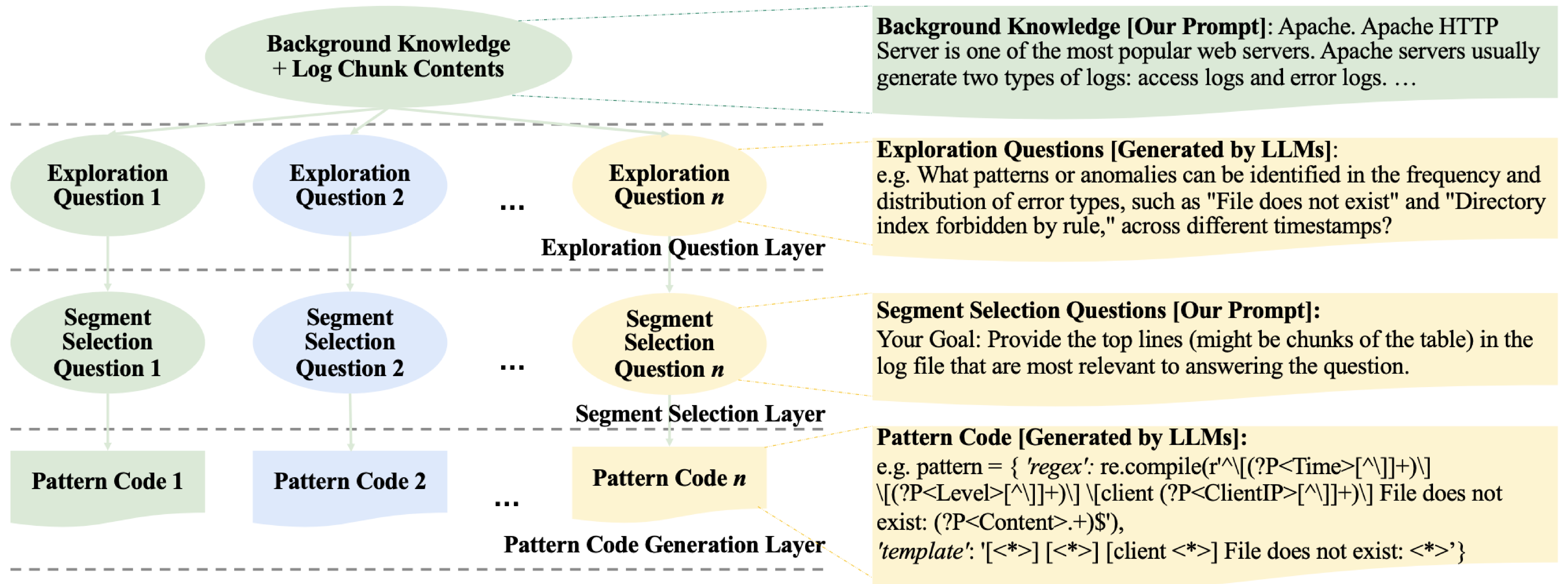
LLM assisted rewrite rule selection and generation

Results: -24.23% area / -12.43% delay vs e-graph only baseline



ADA Example 4: Log Analysis

Agent generates tools/code to extract structure information from log files



Challenges Of ADA

Domain Capability

Question Prompt:

Implement the Verilog module based on the following description. Assume that signals are positive clock/clk edge triggered unless otherwise stated.

Problem Description:

Given an 8-bit input vector [7:0], reverse its bit ordering.

```
module top_module (  
    input [7:0] in,  
    output [7:0] out  
);
```

Canonical Solution:

```
    assign {out[0],out[1],out[2],out[3],out[4],  
           ↪ out[5],out[6],out[7]} = in;  
endmodule
```

VerilogEval (2023)

70+% pass rate

ID	Category Description	Volume		Tokens Mean/Max (k)	
		Non-Agentic	Agentic	Non-Agentic	Agentic
Code Generation					
cid02	RTL - Code Completion	94	0	1.5/4.5	–
cid03	RTL - Natural Language Spec to Code	78	37	1.2/6.9	2.7/7.9
cid04	RTL - Code Modification	56	26	2.0/4.6	5.7/19.5
cid05	RTL - Spec to RTL (Module Reuse)	0	26	–	7.4/28.5
cid07	RTL - Code Improvement (Linting/QoR)	41	0	1.9/5.9	–
cid12	Design Verification - Testbench Stimulus Gen.	68	18	1.4/6.2	2.1/4.6
cid13	Design Verification - Testbench Checker Gen.	53	18	2.8/7.3	4.5/10.7
cid14	Design Verification - Assertion Generation	68	30	2.6/7.5	4.8/14.6
cid16	Design Verification - Debugging / Bug Fixing	36	11	2.3/6.5	3.9/14.5
Code Comprehension					
cid06	Correspondence - RTL to/from Specification	34	0	1.6/5.5	–
cid08	Correspondence - Testbench to/from Test Plan	29	0	3.1/6.1	–
cid09	Question & Answer - RTL	34	0	1.1/5.0	–
cid10	Question & Answer - Testbench	26	0	3.6/4.8	–
	Total # of Problems	617	166		

CVDP (2025)

10%-40% pass rate

Train a Domain-Adapted Model

ScaleRTL: Scaling LLMs with Reasoning and Test Time Compute

Phase 1: Fine-Tuning on Large-Scale CoT Data

Training Sample

Training Sample

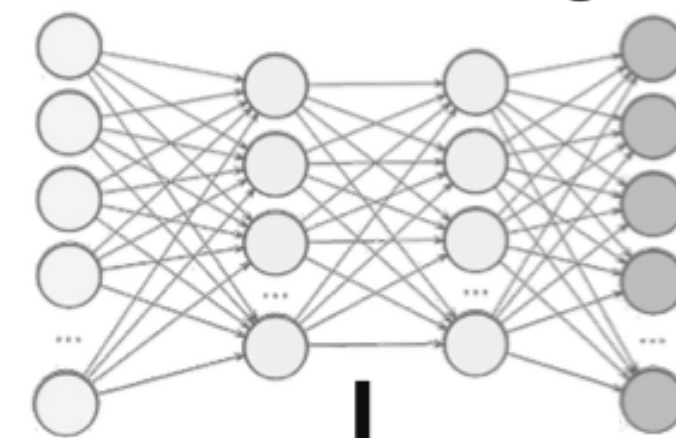
Training Sample

Specification
Build a counter that counts from 0 to 999, inclusive, with a period of 1000 cycles. The reset input is active high synchronous, ...

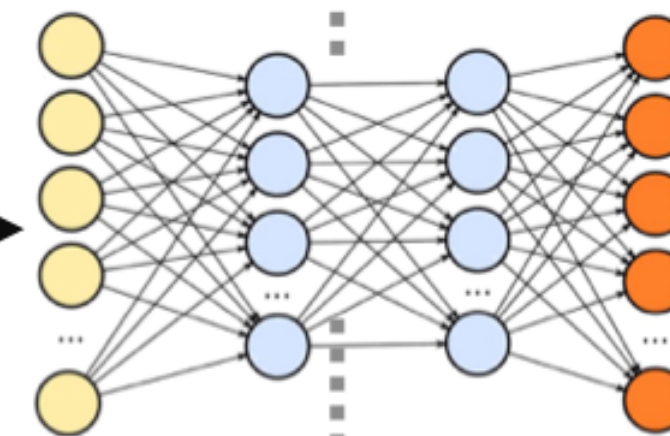
CoT Reasoning Trace
<think>
Okay, so I need to design a counter that goes from 0 to 999 ...
* First, the counter has to increment on every clock cycle. So ...
* That makes sense. But how are the bits handled? Since the output ...
* Wait, when you write 999, ...
* Let me think about edge cases. What happens when q is 999 ...
* Testing scenarios: reset is high, then q stays 0. When reset is low, ...
Yes. The code should handle all that.
</think>

Code Solution
<answer>
always @(posedge clk) begin
 if (reset) begin
 q <= 10'b0;
 end else begin
 if (q == 999) begin
 q <= 0;
 end else begin
 q <= q + 1;
 end
 end
end
</answer>

General Reasoning Model



ScaleRTL
(capable of RTL reasoning
via CoT fine-tuning)



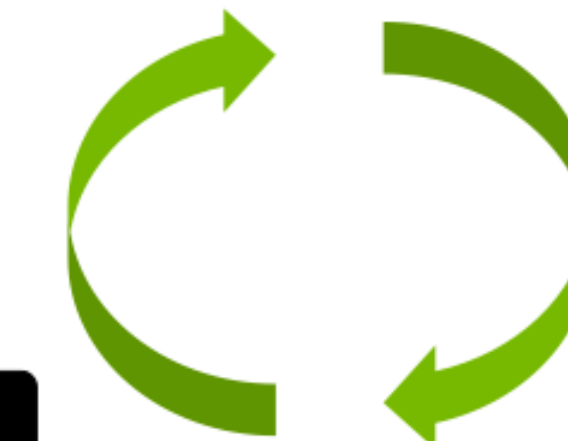
Phase 2: RTL Reasoning with Test-Time Scaling

Test Sample

Specification
Create a set of counters suitable for use as a 12-hour clock
(with am/pm indicator) ... hh, mm, and ss are two BCD (Binary-
Coded Decimal) digits each for hours (01-12), minutes (00-59),
and seconds (00-59). Reset is the active high synchronous ...

Corrective Prompt for Reasoning Extension

<think>
Alright, let's tackle this problem. The goal is ..., let's break it down step by step
* First, I need to model the seconds, minutes, and hours counters, ...
* Starting with the reset condition: when reset is high, the time should be ...
* Wait, but how to handle the BCD increment? Let's think ...
* But this could get complex. Alternatively, handle the entire increment as ...
* Wait, but how to handle BCD increment for hours. For example, ...
* ...
Yes. This covers the necessary transitions. So the code should work.
Wait, my reasoning must be incorrect. Let's check the reasoning rules: {Reasoning Rules}. Wait, I did not follow the rules. Let's try again.

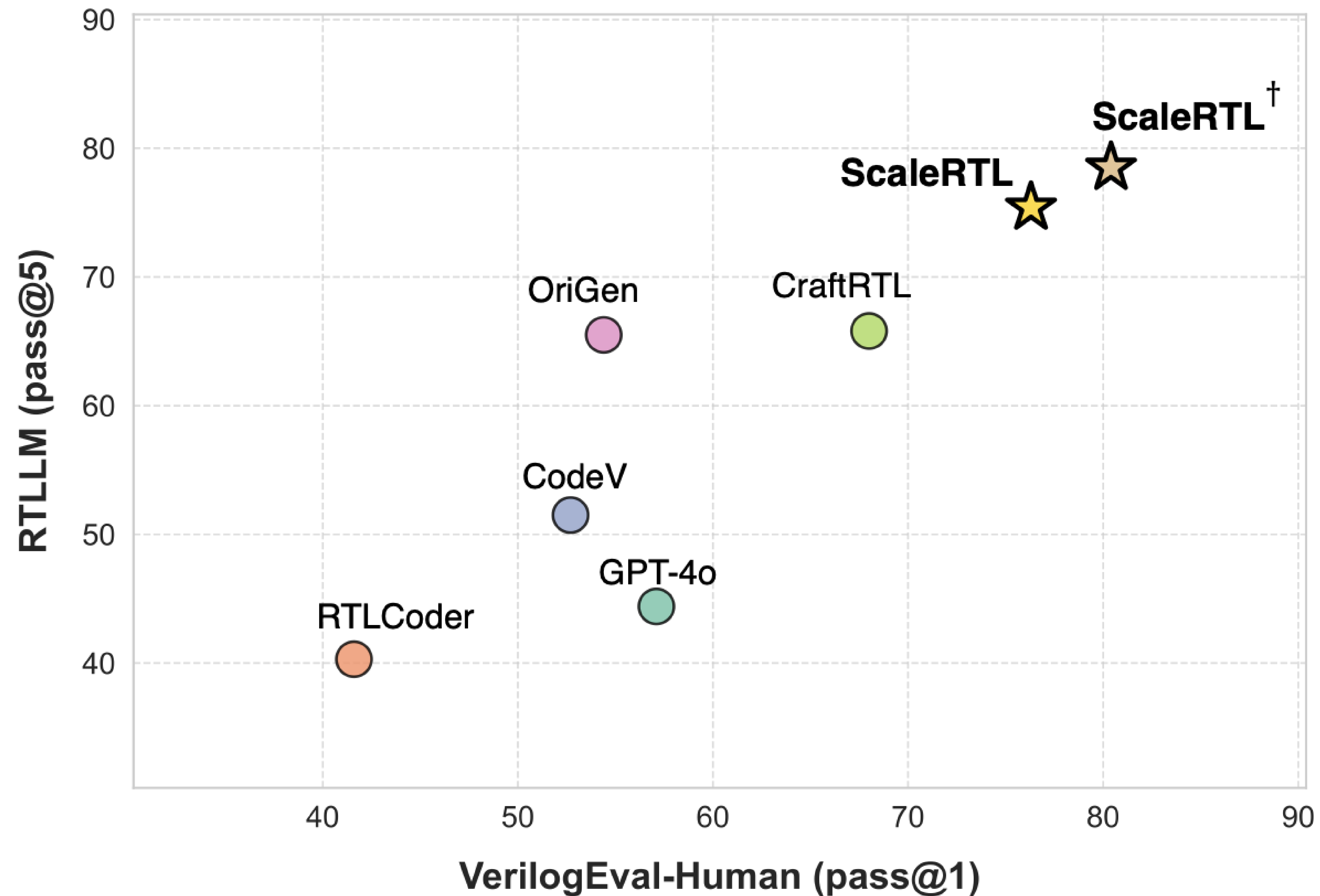


Iteratively extending
reasoning process
via corrective prompt

Model Output

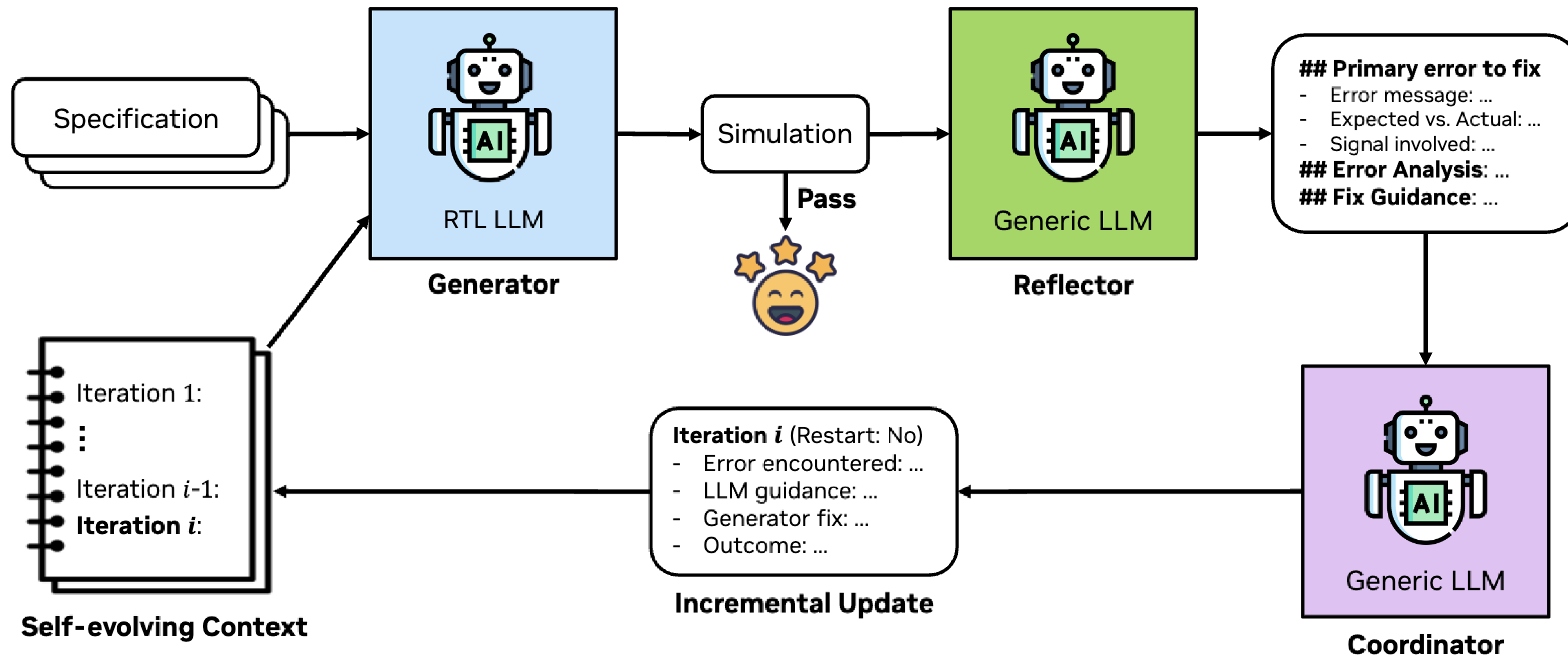
<think>
Alright, let's tackle this problem. The goal is ..., let's break it down step by step
* First, I need to model the seconds, minutes, and hours counters, ...
* Starting with the reset condition: when reset is high, the time should be ...
* Wait, but how to handle the BCD increment? Let's think ...
* But this could get complex. Alternatively, handle the entire increment as ...
* Wait, but how to handle BCD increment for hours. For example, ...
* ...
Yes. This covers the necessary transitions. So the code should work.
</think>
<answer>
[Code is omitted due to space constraints]
</answer>

ScaleRTL (32B) was the SOTA small LLM on VerilogEval

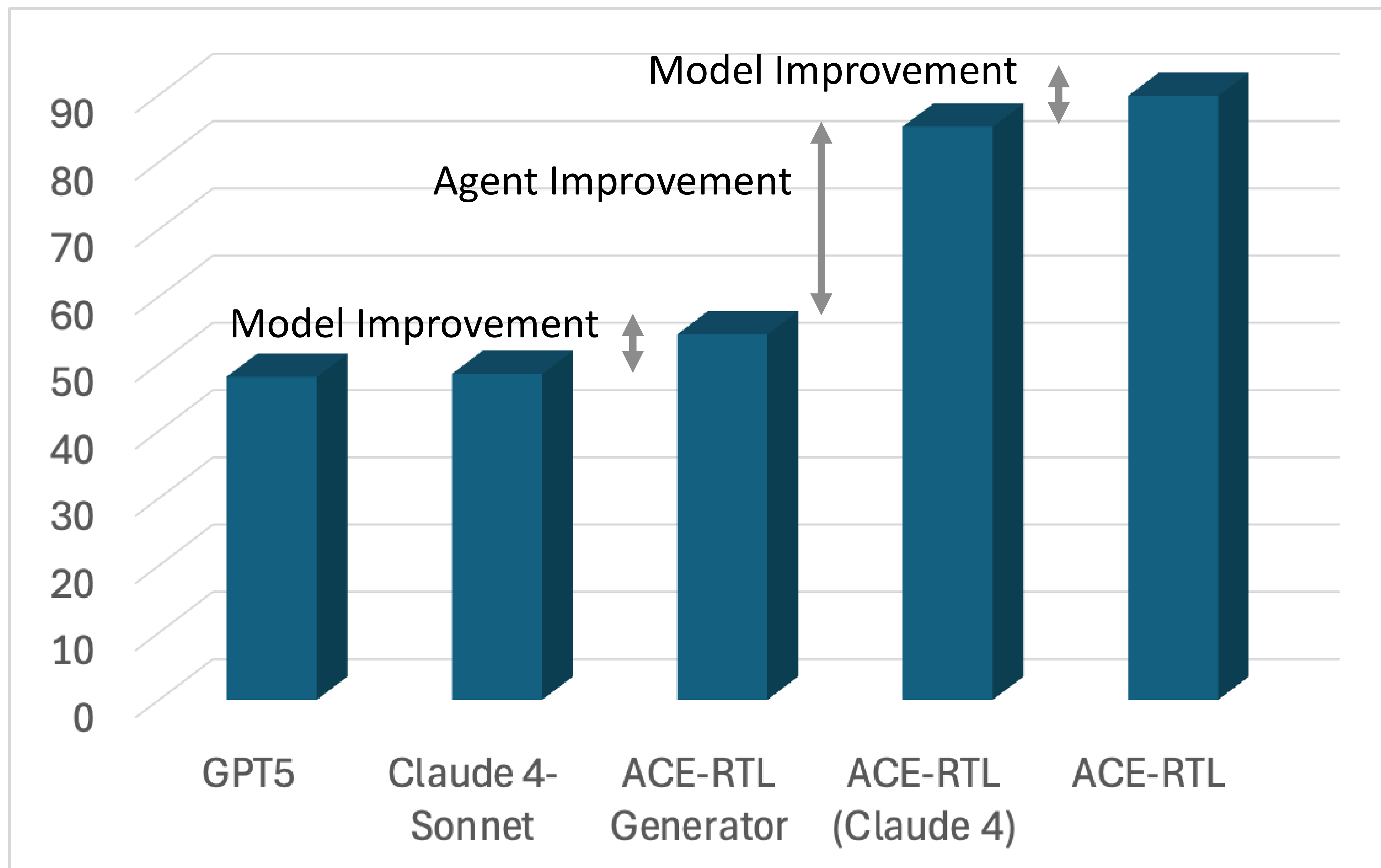


How to Use Domain-Adapted Models in Agent

ACE-RTL: Hybrid Models and Agentic Context Evolution

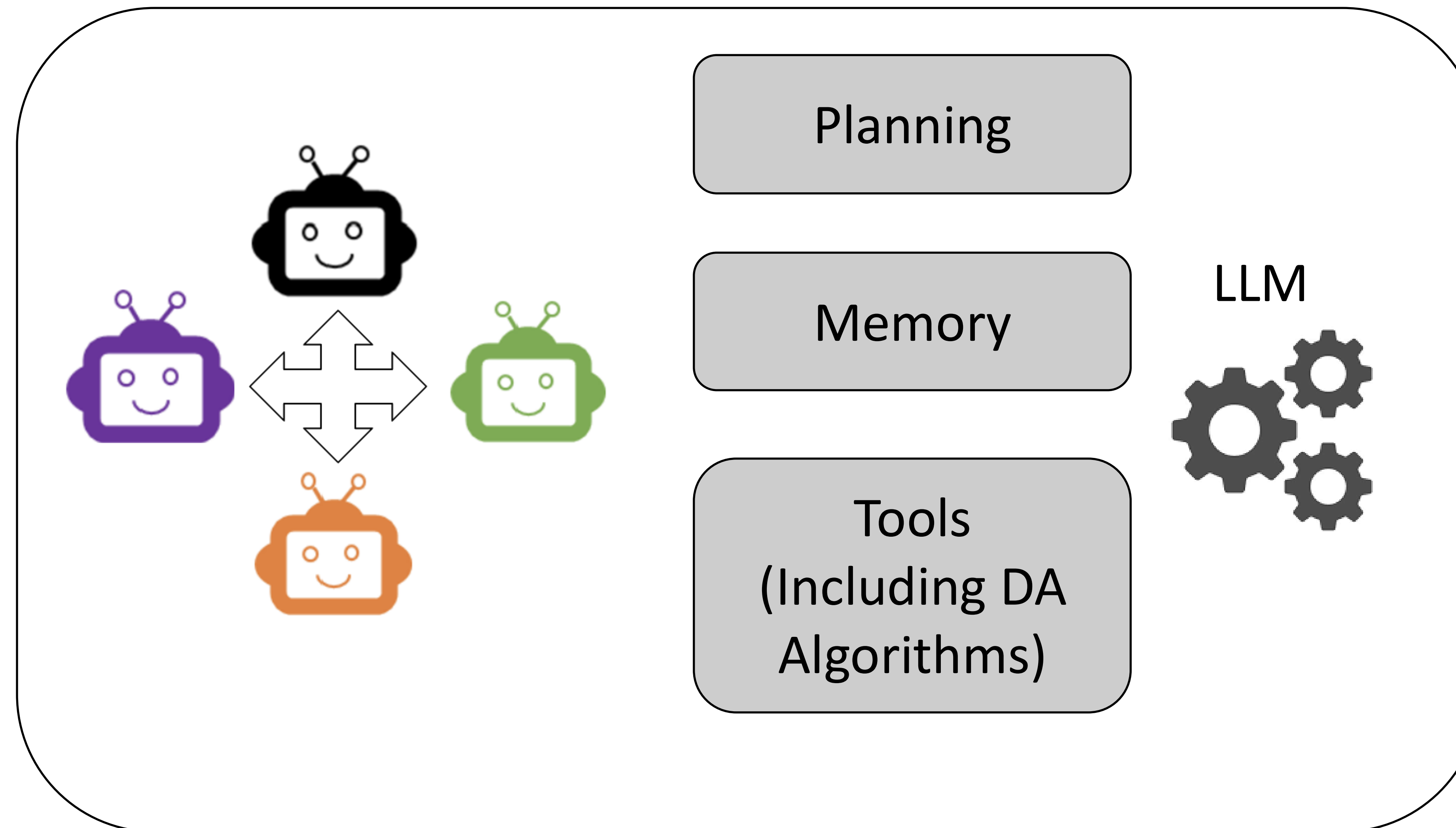


Achieved 88.9% Pass Rate Across Four CVDP RTL Coding Tasks



Challenges of ADA

Performance Optimization

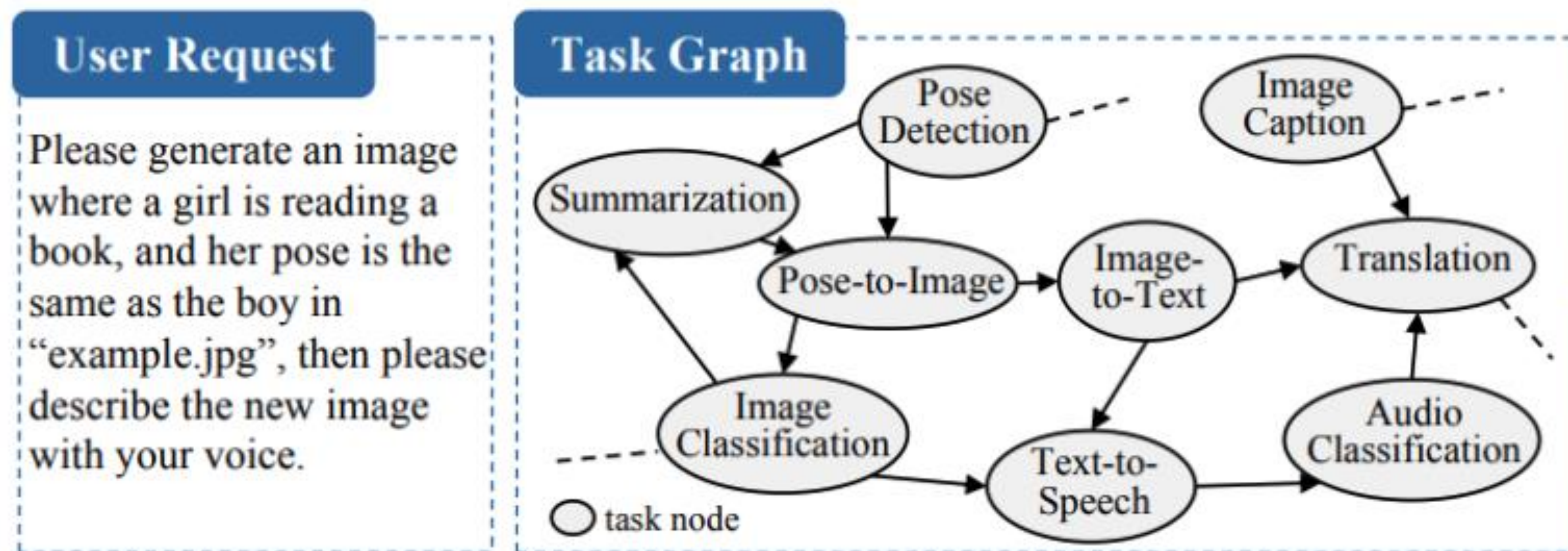


How to optimize the entire system?

Make Good Plans

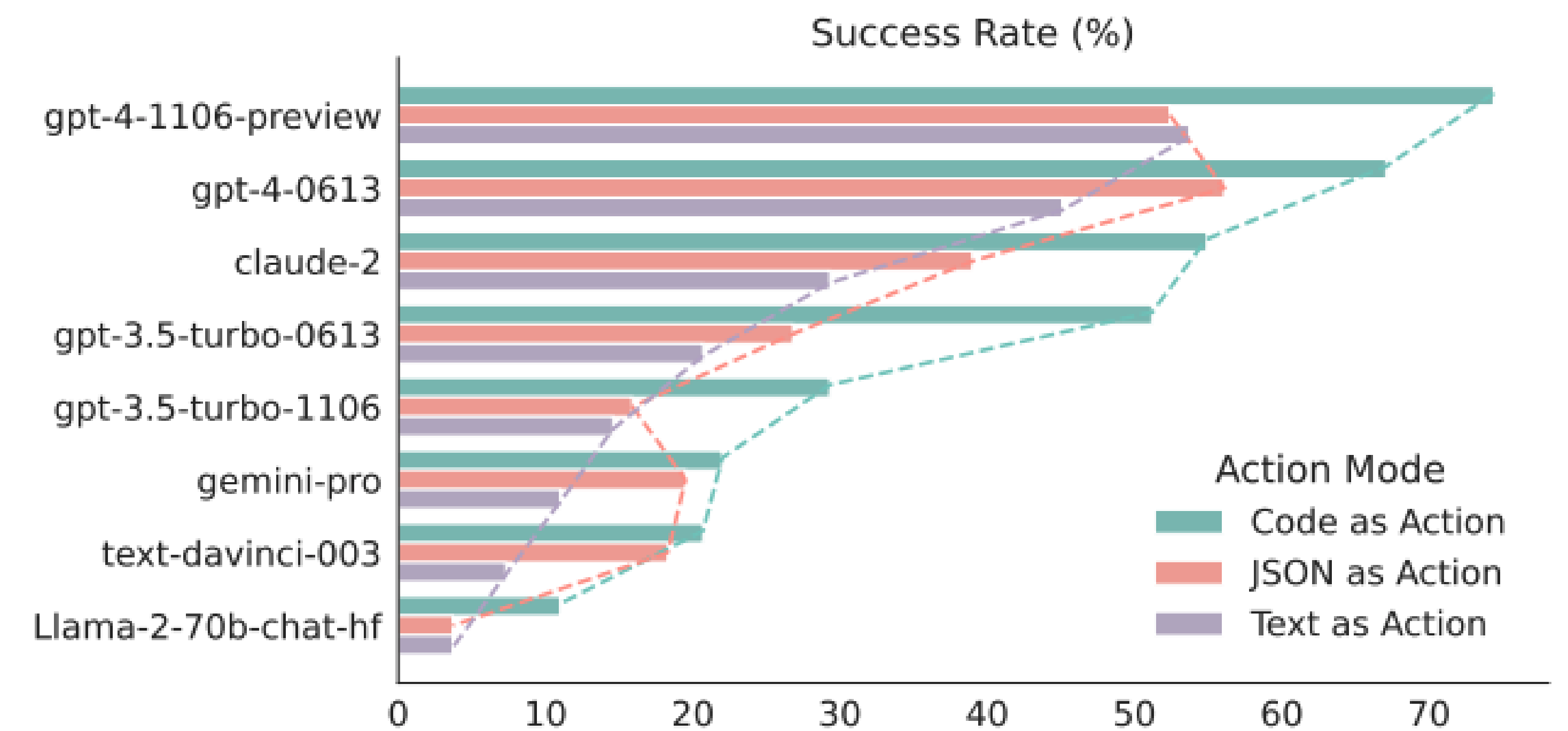
Favors structured plans

Graph as Plan



HuggingGPT

Code as Plan



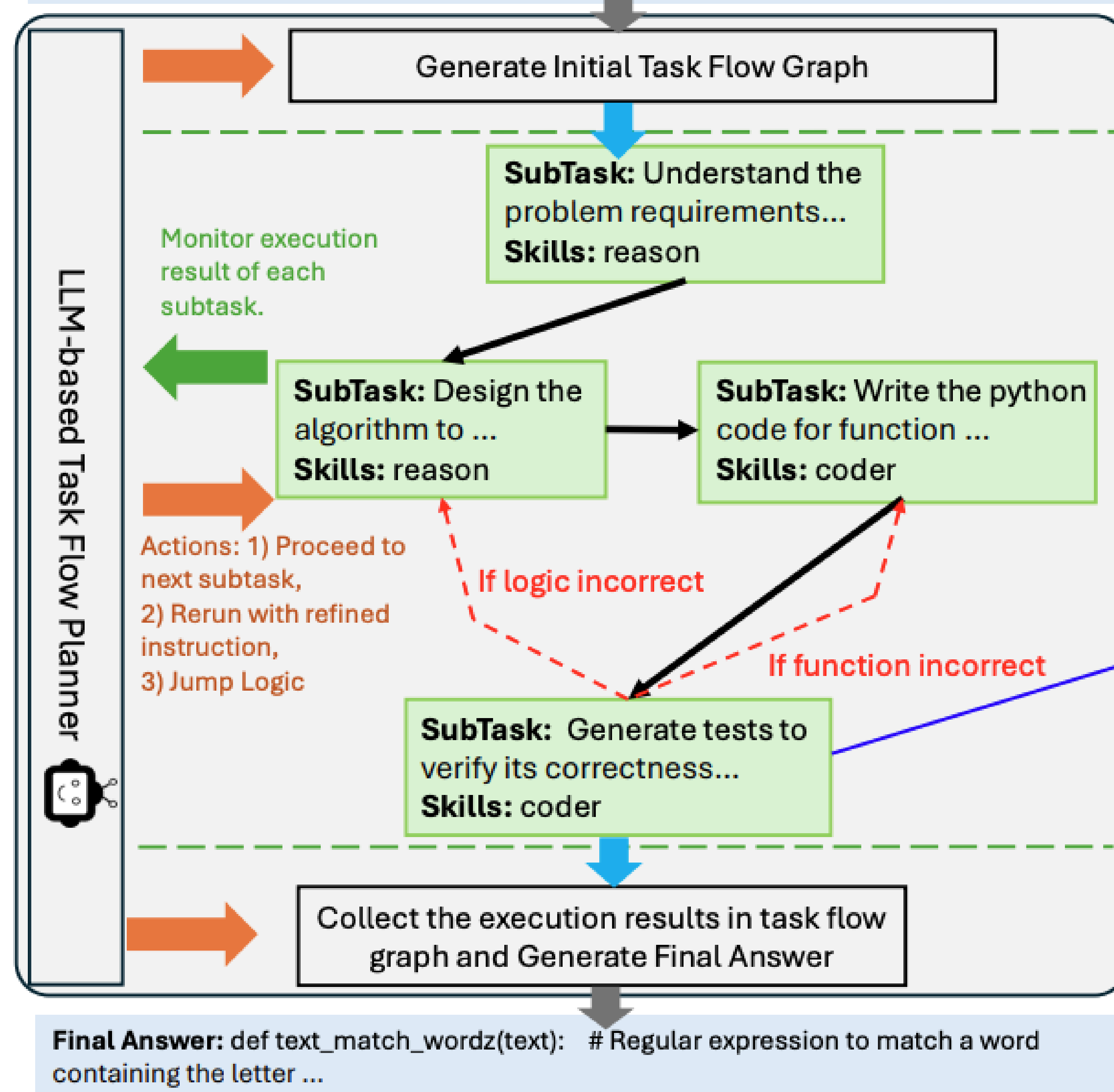
CodeAct

Polymath Agent

Hierarchical Planning with graph and code plans

(a) An example of task flow graph solving

Task Prompt: You are a Python Coding expert. Complete the text_match_wordz function and return the completed code only...



(b) An example of subtask code-represented workflow

SubTask: Generate tests to verify its correctness...

```
def workflow(instruction: str):
    instruction = "### Instruction Background ###\n" + instruction + "\n"
    <<<<<<< SEARCH
    ... # your designed workflow here.
    =====

    import json

    # Task_3: Test the text_match_wordz function
    # Test inputs for the function
    test_cases = [
        "apple",
        "zebra",
        "buzz",
        "fizz",
        "hello"
    ]
    # Expected outputs: [False, True, True, True, False]
    expected_outputs = [False, True, True, True, False]

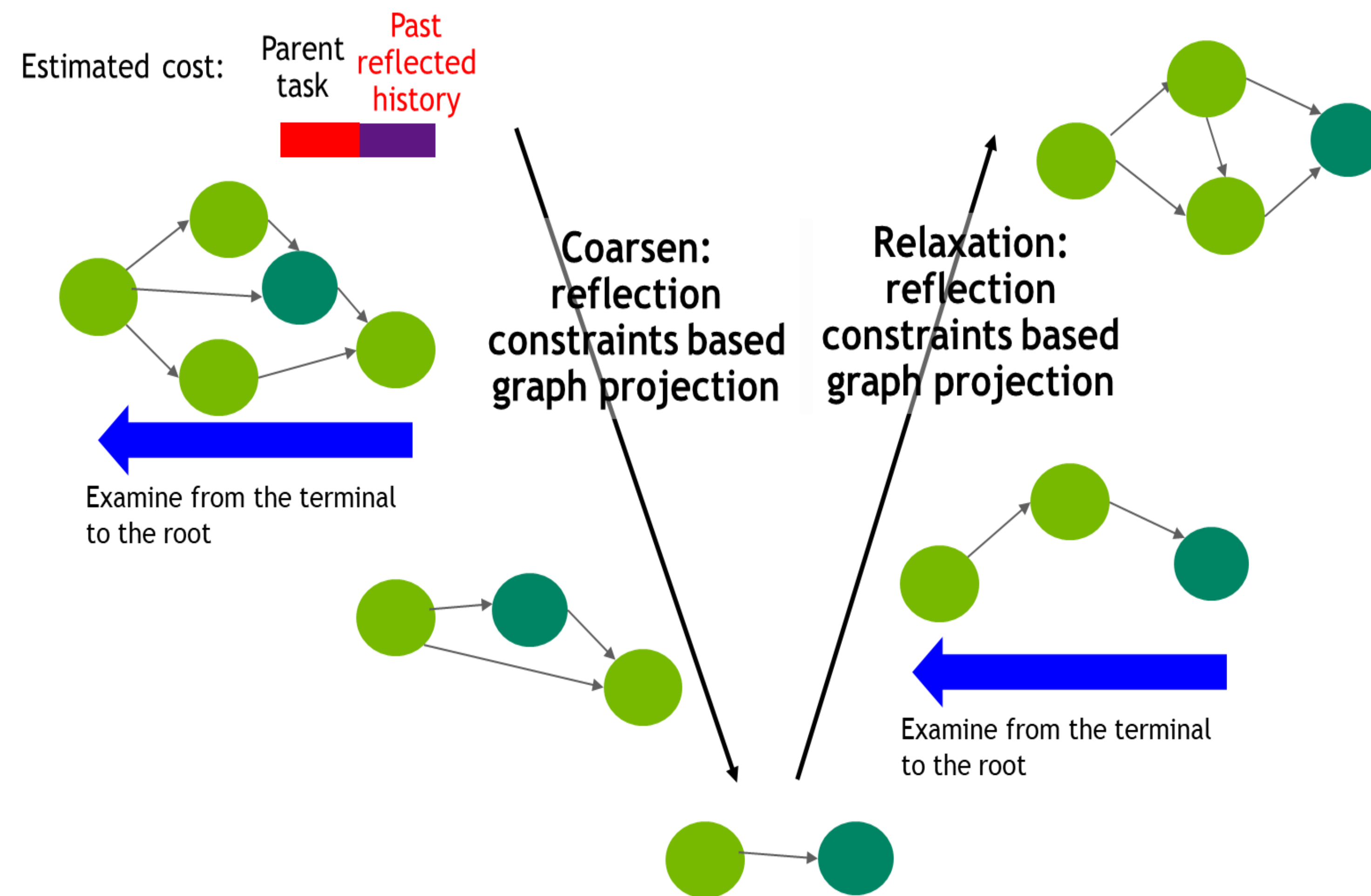
    # Chat with coder to run tests on function
    coder_instruction = f"Test the text_match_wordz function with inputs
{test_cases}.\n"
    coder_instruction += "Ensure the function returns the correct boolean values
indicating the presence of 'z' in words.\n"
    coder_instruction += "Return True for words containing 'z' and False
otherwise.\n"
    response = coder_assistant.initiate_chat(message=instruction +
coder_instruction)

    # Output the test results
    >>>>>>> REPLACE
    return response
```


Optimize the Plan

State-of-the-art results on multiple QA, coding and math benchmarks

Top-level Divide & Conquer: Manageable Subtask



Multi-grid based optimization based on
LLM-as-judge

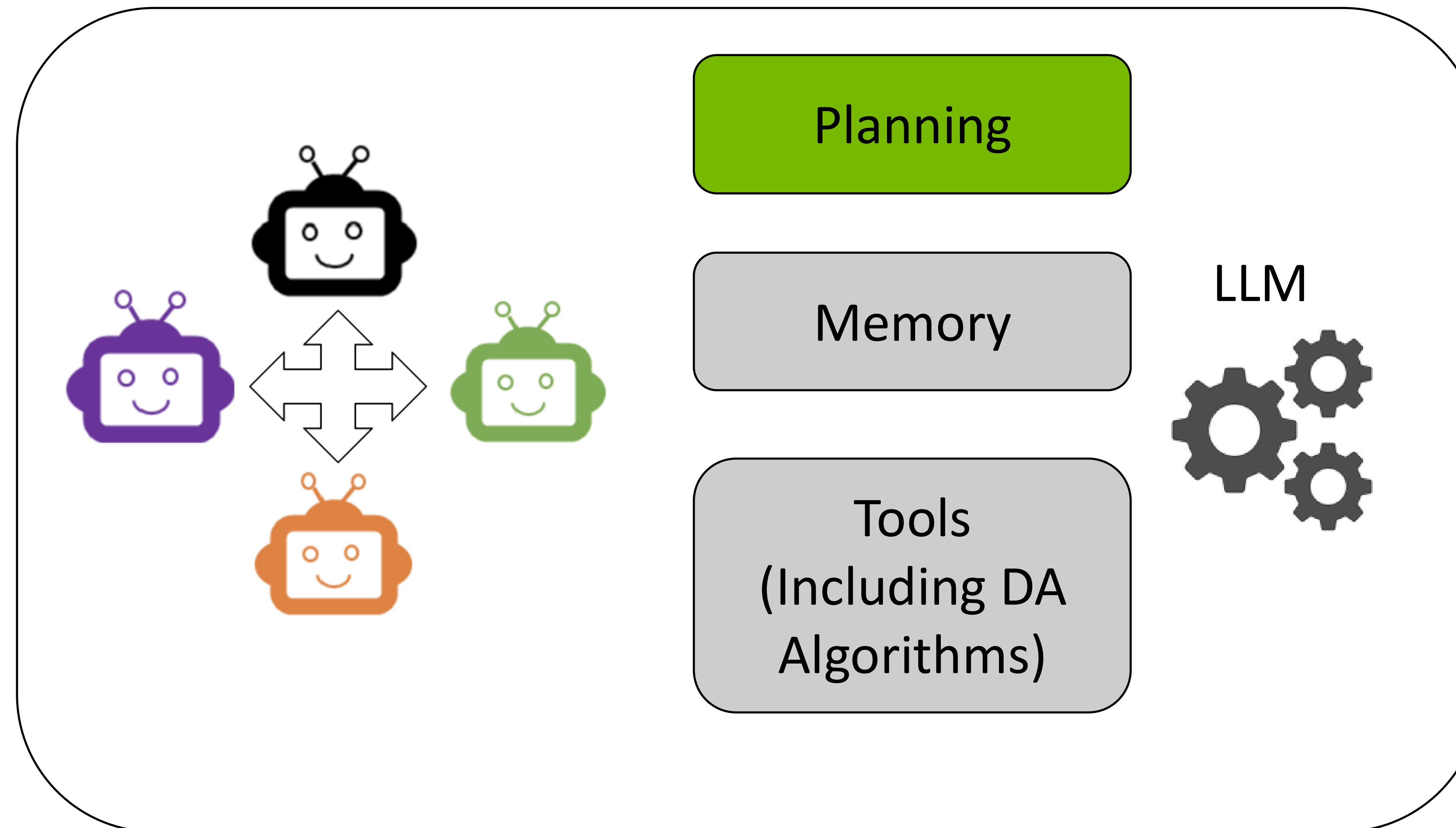
Subtask/Coding-level (Evolutionary Alg.) Optimize subtask execution & performance

```
39 # EVOLVE-BLOCK-START
40 def workflow(instruction: str):
41     """
42     Workflow to generate the 'drop_empty' function following the instruction input.
43     """
44     # Send the instruction directly to the coder assistant
45     coder_response = coder_assistant.initiate_chat(message=instruction)
46     coder_output = extract_output(
47         response=coder_response,
48         post_process_func=coder_description["post_process_func"]
49     )
50     # Return the coder's output directly
51     return coder_output
52 # EVOLVE-BLOCK-END
53
54 # Getting the strong reason debug assistant response
55 # Passing all reasoning to the coder assistant for testing implementation
56 strong_reason_debug_summary = extract_output(
57     response=strong_reason_debug_assistant_response,
58     post_process_func=reason_agent_description["post_process_func"]
59 )
60
61 # Sending the validated reasoning to coder assistant for testing implementation
62 coder_task = instruction + "\n" + "### Test Cases for 'drop_empty' ###" + strong_reason_debug_summary
63
64 coder_response = coder_assistant.initiate_chat(message=coder_task)
65 coder_output = extract_output(
66     response=coder_response,
67     post_process_func=coder_description["post_process_func"]
68 )
69
70 # Final string output as response
71 response = json.dumps({
72     "task": "Test the 'drop_empty' function with various dictionaries.",
73     "testing_results": coder_output
74 }, indent=4)
75
76 return response
77 # EVOLVE-BLOCK-END
```

Self-Reflection Guided Evolution

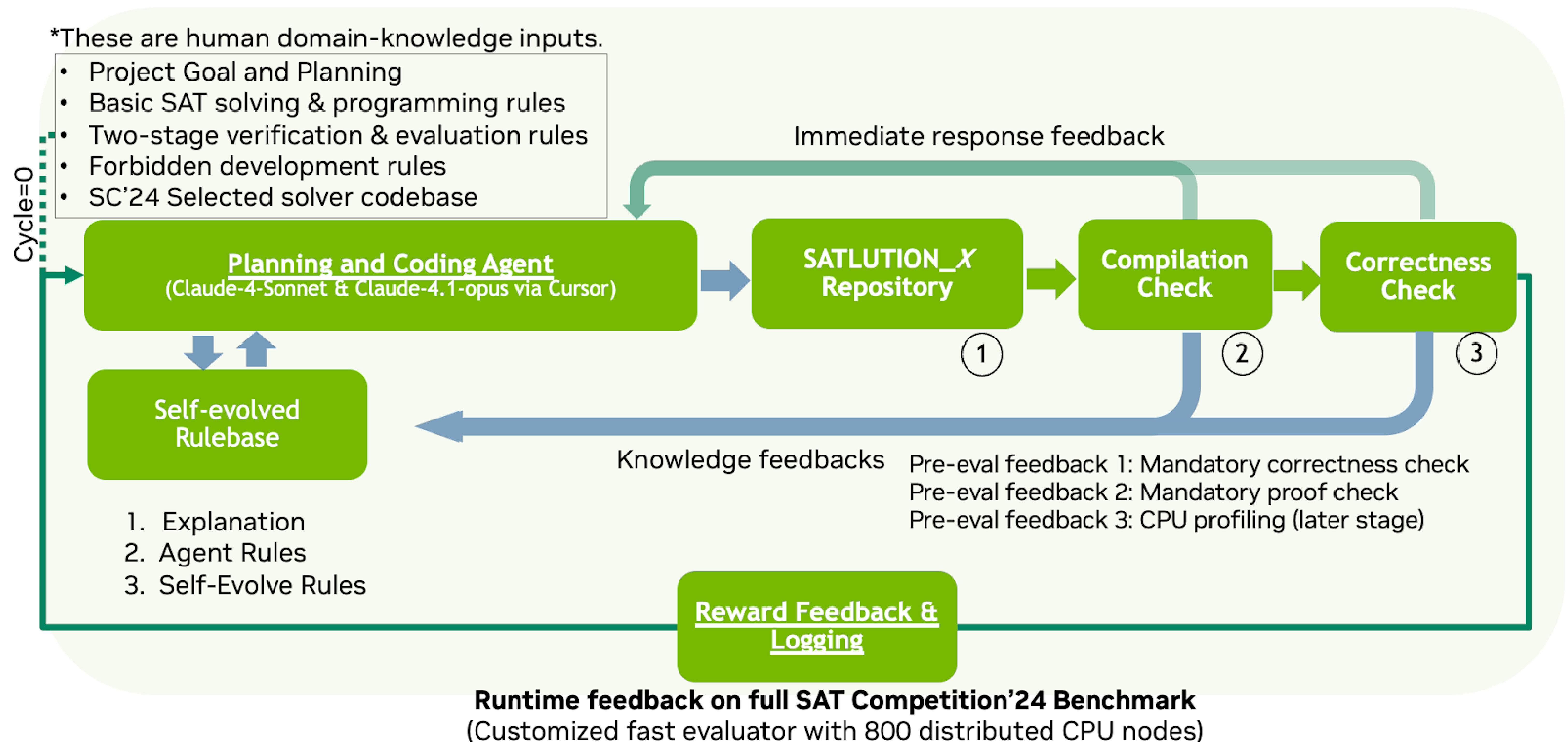
Challenges of ADA

Performance Optimization



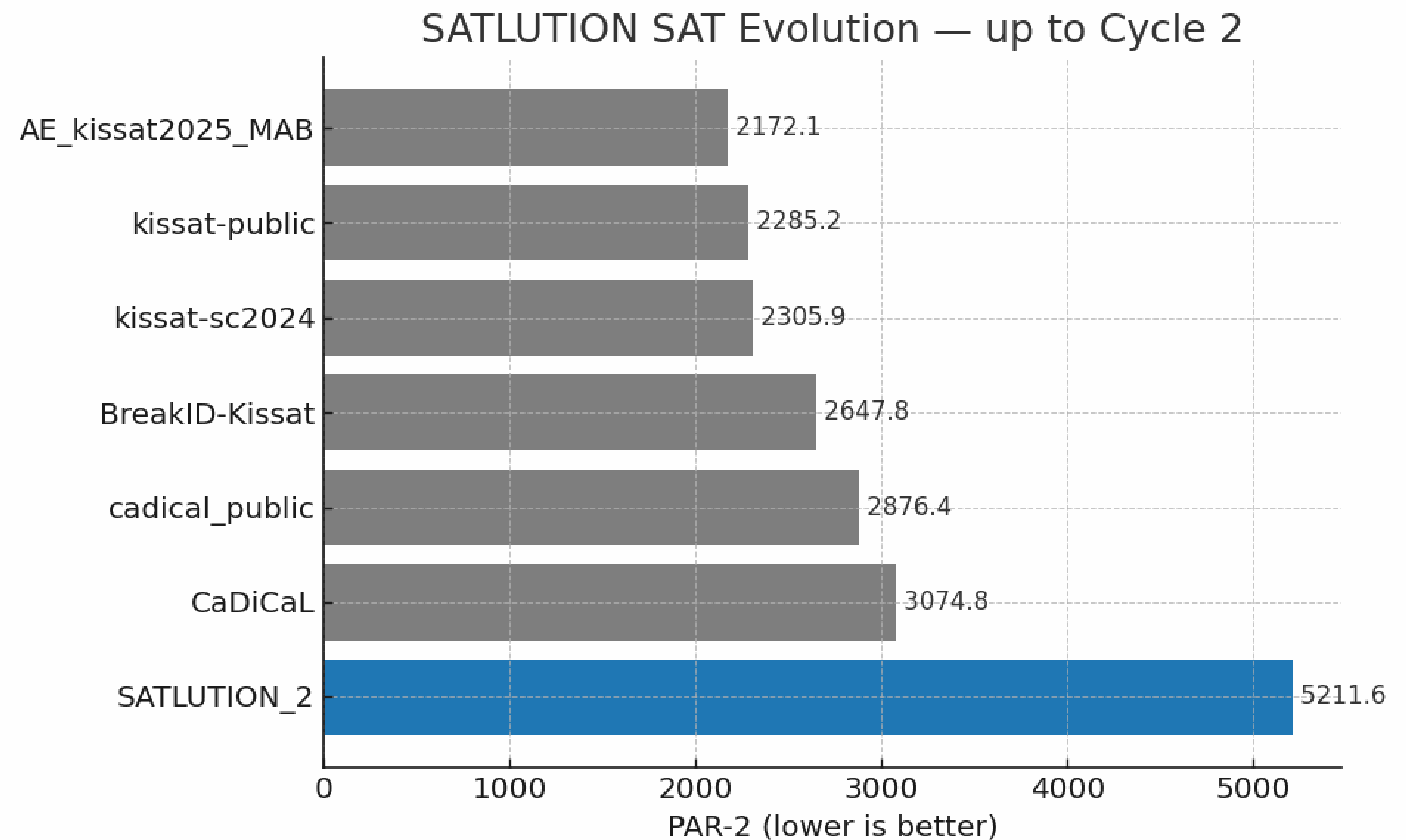
How to optimize the entire system?

SATLUTION: Evolve SAT Solvers



SATLUTION : 10+% better than 2025 SAT Competition winners

Start from the winning solvers
in 2024 SAT competition
Improved performance by
18% in a month of evolution



Toward Autonomous Design

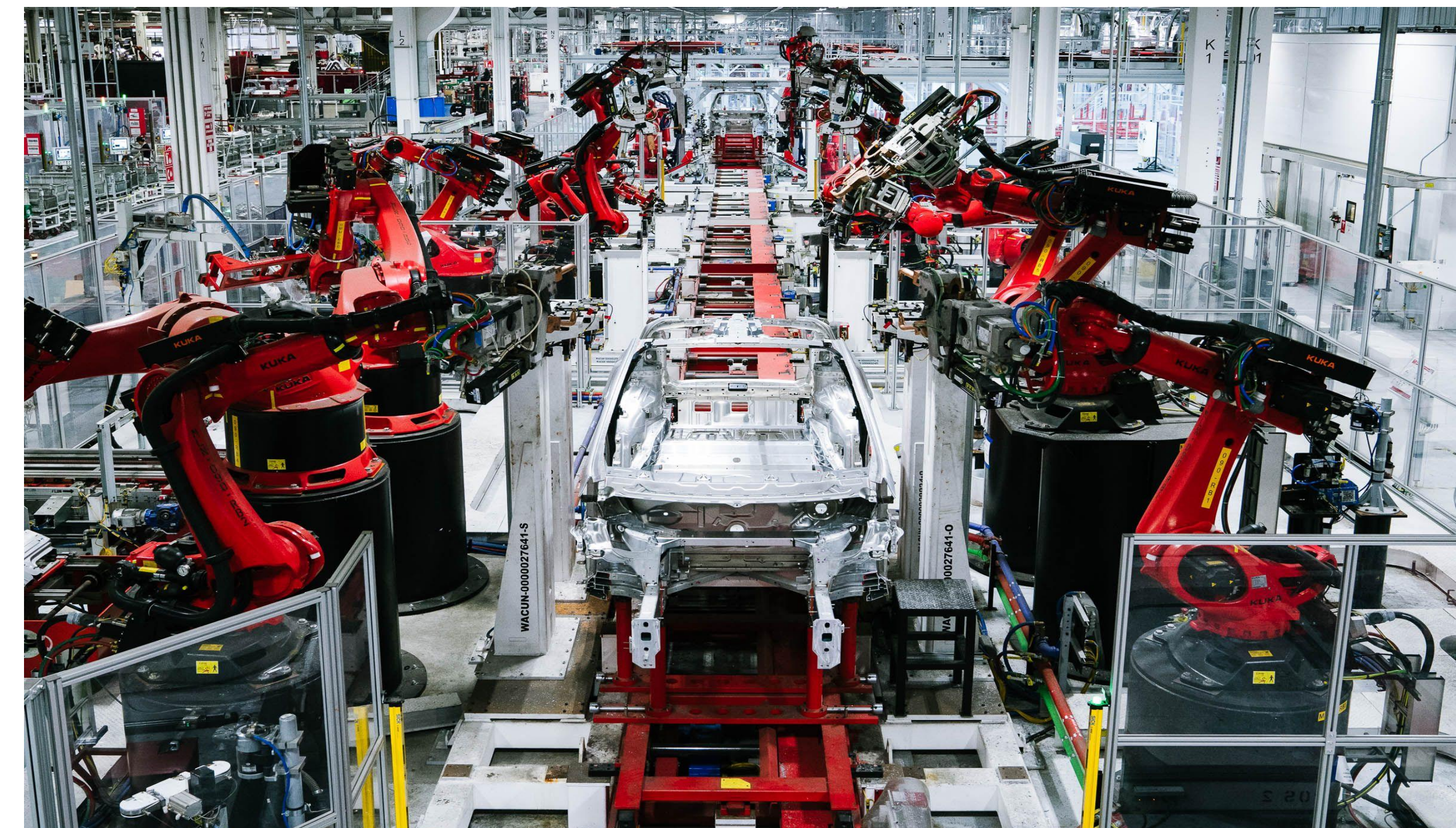
Agentic Design Automation will lead us to autonomous design

Domain capabilities and optimality are the key challenges to overcome

Self-improving capability is very important: learn while doing

Need Agent-native tools to make agents more capable

Scaling of Agent and Agent-native tools will bring us to the future



AGENTRYS is an applied research team
building the future of chip design.

